WEST UNIVERSITY OF TIMIŞOARA
FACULTY OF COMPUTER SCIENCE
BACHELOR STUDY PROGRAM: COMPUTER SCIENCE IN ENGLISH

BACHELOR THESIS

SUPERVISOR:
Lect. Dr. Cristian Cira

GRADUATE:
Madalina-Gabriela Gorcea

TIMIŞOARA
2026

WEST UNIVERSITY OF TIMIŞOARA
FACULTY OF COMPUTER SCIENCE
BACHELOR STUDY PROGRAM: COMPUTER SCIENCE IN ENGLISH

Exploring the Autonomous Behavior of Large Language Models in Simulated Environments

SUPERVISOR:
Lect. Dr. Cristian Cira

GRADUATE:
Madalina-Gabriela Gorcea

TIMIŞOARA
2026

Abstract

The purpose of this thesis is to explore the behavior of large language models (LLMs) when placed in a simulated environment. Accelerated development in LLMs has provided abilities to reason, plan, and act in context-rich scenarios using natural language prompts. However, their potential to interact independently within complex and dynamic systems remains largely unexamined. Such systems are represented by video games. This study proposes an experimental framework where a virtual environment and structured contextual information about general and specific elements are provided to an embedded LLM. Based on this context, the LLM has the freedom to decide its next action, affecting future game states, navigation or objects. The study aims to observe patterns of reasoning, goal formulation, and behavioral consistency in the model's decisions.

The research follows an empirical, simulation-based methodology. Key points include game environment selection and setup, by identifying and setting up an open-source game or simulation framework. This environment will be used as the controlled space for testing LLM's decision-making behavior. Another major detail is context extraction and representation achieved by implementing a system that periodically captures and encodes the full game state into structured inputs understandable by the LLM surroundings. LLM action loop integration establishes an iterative loop where, every 5 seconds (one decision loop), the current context is sent to the LLM. In addition, data collection and logging throughout the experiment, all decisions, states, and outcomes are mandatory, to form a detailed behavioral trace of the LLM's autonomous operation. The final data will be analyzed to detect recurring behavioral motifs, goal-directed planning, or random wandering. Analysis and evaluation will be done using qualitative and quantitative metrics, therefore the LLM's performance will be assessed in terms of consistency, coherence, adaptability, and goal formation.

This research-oriented thesis bridges the domains of artificial intelligence, natural language understanding, and simulation-based testing, offering insights into how LLMs perceive and act within complex virtual worlds.

Rezumat

Scopul acestei lucrări este de a analiza comportamentul modelelor lingvistice mari (LLM) atunci când acestea sunt plasate într-un mediu simulat. Dezvoltarea accelerată a modelelor lingvistice mari a condus la apariția unor capacități avansate de raționament, planificare și acțiune în scenarii bogate contextual, prin intermediul instrucțiunilor formulate în limbaj natural. Cu toate acestea, potențialul acestor modele de a interacționa independent în cadrul unor sisteme complexe și dinamice rămâne, în mare măsură, insuficient explorat. Un exemplu reprezentativ pentru astfel de sisteme este oferit de jocurile video. Prezenta lucrare propune un cadru experimental în care unui LLM integrat îi sunt furnizate un mediu virtual și informații contextuale structurate despre elemente generale și specifice ale acestuia. Pe baza acestui context, modelul are libertatea de a decide următoarea acțiune, influențând stările viitoare ale jocului, navigarea sau obiectele din mediu. Studiul urmărește observarea tiparelor de raționament, a formulării obiectivelor și a consistenței comportamentale în deciziile modelului.

Cercetarea urmează o metodologie empirică, bazată pe simulare. Aspectele esențiale includ selecția și configurarea mediului de joc, prin identificarea și pregătirea unui joc open-source sau a unui cadru de simulare. Acest mediu va fi utilizat ca spațiu controlat pentru testarea comportamentului decizional al LLM-ului. Un alt element important îl reprezintă extragerea și reprezentarea contextului, realizate prin implementarea unui sistem care capturează periodic starea completă a jocului și o codifică sub forma unor intrări structurate, inteligibile pentru LLM. Integrarea buclei de acțiune a LLM-ului stabilește un proces iterativ în care, la fiecare 5 secunde, corespunzător unei bucle decizionale, contextul curent este transmis modelului. În plus, colectarea și înregistrarea datelor pe parcursul experimentului sunt obligatorii, incluzând toate deciziile, stările și rezultatele, pentru a forma o urmă comportamentală detaliată a funcționării autonome a LLM-ului. Datele finale vor fi analizate pentru identificarea unor motive comportamentale recurente, a planificării orientate spre obiective sau a deplasării aleatorii. Analiza și evaluarea vor fi realizate prin utilizarea unor metrici calitative și cantitative, performanța LLM-ului fiind apreciată din perspectiva consistenței, coerenței, adaptabilității și formării obiectivelor.

Această lucrare cu orientare de cercetare creează o punte între domeniile inteligenței artificiale, înțelegerii limbajului natural și testării bazate pe simulare, oferind perspective asupra modului în care LLM-urile percep și acționează în cadrul unor lumi virtuale complexe.

Contents

Chapter 1

Introduction

1.1 Background and motivation

For the domestic user, a large period of time LLMs were just plain text-generators, given the proper context and relevant details a coherent human-like text was provided, but recent developments show several emergent capabilities [14]. The thesis studies these new abilities by using LLMs as agents.

An LLM is considered an agent when it is integrated in a decision-action loop where it receives details from the environment, reasons, selects, and executes the actions which can influence the future state of the environment.

Theoretically, the agent and loop could be placed in a physical world. However, such approaches prove themselves as costly and time consuming, therefore the choice of simulated environments to test the capabilities of LLMs is not a coincidence. Such habitats have been used to create believable proxies of human behavior [6].

The thesis strives to understand emergent patterns in autonomy, to provide solid data on behavior and reliability. Behavior represents sequences of decisions that interact, therefore understanding it can reveal planning, short-term or long-term, patterns and possible issues such as inconsistent reasoning or unexpected strategies. The problems presented above can represent risks in regards to safety. Reliability is based on the agent's ability to maintain coherent and consistent decision-making in varying contexts.

1.2 Problem statement

Whilst modern day LLMs can perform various tasks, ranging from reasoning to using tools [15, 8], their long-term autonomous behavior and consistency remain largely a mystery. The thesis aims to shine a little light on the less understood parts of agents, in order to appreciate the predictability in behavior and deployment limitations in interactive systems.

1.3 Objectives and questions

The objective of this study is to investigate the autonomous behavior of an LLM in a simulated environment where the focus falls on how the model perceives context, decision making and adapting rather than optimization of task performance.
For this study to succeed the following objectives are necessary:

Experimental framework Finding and adapting an existing framework to integrate an LLM into a simulated habitat.

Represent the environment Enable the LLM to understand and reason about the surroundings using supported inputs.

Action selections Allow the LLM to take various actions without explicit reward or predefined goal.

Logging Collect relevant logs containing decisions, outcomes and changes in environment.

Result analysis Interpret the obtained logs to identify patterns or random exploration.

Based on the objectives defined the questions that the study aims to answer are:

- How does an agent behave when granted permanent autonomy in a simulated environment?
- What are the visible patterns in the behavior of the LLM?
- Does the LLM present consistent and coherent decision-making during extended periods?
- What abilities does the LLM exhibit in order to adapt to changes in environment?

The objectives and the proposed questions are guiding the thesis.

1.4 Limitations

This study is subject to several limitations that influence both the scope of the experiment and the interpretation of the results. Such limitations are:

Five runs per configuration each configuration was executed five times on a fresh world, so the reported aggregates reflect repeated behaviour rather than a single trajectory.

No visual perception as the agent perceives the world only through structured textual observations derived from the Mineflayer protocol. This means that its understanding of the environment is limited to the information included in the observation pipeline and to fixed scanning radii.

Fixed action space as the agent is not developing new abilities, but it is provided both low-level primitives and high-level macros to use in a run. This moves the focus of the study on the behavior, reasoning and goal definition rather than skill acquisition.

Self-defined goals by the model and do not always guarantee alignment with the actions that follow. Some analysis signals, such as behavioral thrashing, are based on heuristic thresholds rather than externally validated measures.

Taken together, these limitations do not invalidate the study, but they define its scope clearly: the thesis should be understood as a controlled exploratory investigation of LLM behavior in a simulated environment, rather than as a definitive benchmark of autonomous game-playing performance.

1.5 Thesis structure

This section is dedicated to providing summarized information regarding individual chapters facilitating the retrieval of relevant data for the reader's use case.

Chapter 1 Provides the background, motivation and skeleton of the study and introduces the theoretical foundation providing the reader the tools necessary for understanding the current thesis.

Chapter 2 Presents approaches studied before and relevant related works which will be referenced and taken into account the entirety of the study as well as the research approach this thesis relies on.

Chapter 3 Contains a detailed overview of the experimental framework used.

Chapters 4, 5 Provides all the details needed regarding the implementation of the LLM and framework.

Chapter 6 Present the experiments done and the analysis of the results.

Chapter 7 Is the wrap-up of this study and its final or open points.

Chapter 2

Theoretical Background State of the Art

Large Language Models (LLMs) are a class of neural network models designed to process and generate natural language text. Most modern LLMs are based on the Transformer architecture, which relies on self-attention mechanisms to model contextual relationships between tokens in a sequence[12].

Recent advances in model scale and training methodologies have led to significant improvements in reasoning, instruction following, and contextual understanding [3]. Therefore, LLMs are much more than simple text generators, turning incredibly fast into systems capable of performing complicated tasks, such as planning, explaining and deciding.

2.1 Autonomous Agents

An autonomous agent is commonly defined as an entity that perceives its environment through observations and acts upon that environment in pursuit of objectives [7].

The embedding of an LLM within such a loop enables the study of autonomous behavior driven by language-based reasoning rather than optimization. This concept is the core of my thesis.

2.2 Reinforcement Learning vs. Language-Based Agents

Reinforcement Learning (RL) is a dominant paradigm for training autonomous agents, where behavior is shaped through interaction with an environment and guided by a reward function [11].

In contrast, a language-based agent relies on a large language model as its primary decision-making component where the agent selects actions by reasoning over observations represented in natural language or structured text leveraging the implicit knowledge and reasoning capabilities encoded within pretrained language

models [15, 6].

Each agent excels in their own environment, RL are proficient in habitats with clear definition of objectives and rewards, while language-based agents perform well in open-ended settings with ambiguous or emergent goals. Furthermore, language-based agents offer greater interpretability, as their decision processes can be expressed and analyzed through natural language explanations [15, 1].

2.3 Emergent behavior

Emergent behavior refers to complex patterns or strategies that arise from the interaction of simpler components, without being explicitly programmed [4].

Emergence has been observed in various forms in multi-agent reinforcement learning, self-organizing systems and swarm intelligence, decentralized, self-organized systems. The manifestation of emergence can be represented by unexpected strategies, coordination patterns or goal formation.

Although prior research demonstrates the feasibility of using language models as agents, there remains a lack of systematic analysis of their long-term behavior, consistency, and emergent goal formation within complex simulated environments. Existing studies frequently focus on task completion, performance metrics or skill acquisition, but offer little insights into behavioral patterns that develop over extended periods of time.

An evident limitation across current literature is reliance on explicit objectives, rewards, or externally imposed tasks. Voyager [13] defines clear goals and evaluates success based on skill acquisition and progression, while frameworks like ReAct or Reflexion focus on improving task completion by using structured loops for reasoning and feedback [15, 9]. Even in open-ended simulations, Generative Agents [6], agent behavior is guided, be it by implicit social objectives or structured routines. As a result, it is difficult to pronounce whether observed behaviors are genuinely emergent or simply artifacts of the imposed objectives.

Another dimension that lacks is the separation between capability and intention. In most existing approaches, the agent has available actions which are coupled with predefined goals, resulting in evaluation of capabilities based on efficiency to complete a task. This creates a bias toward optimization-driven behavior and limits the ability to study how an agent behaves when no external objective is present. Therefore, questions related to spontaneous exploration, self-initiated structure, or intrinsic behavioral consistency remain insufficiently addressed.

Furthermore, while recent work introduced memory mechanisms, reflection and self-improvement [9, 5], they are evaluated in iterative task settings where only performance improvements can be measured. Not enough attention has been given to how such mechanisms influence behavior in goal-free, continuous environments, where there is no explicit notion of success or failure. Similarly, reinforced

learning approaches provide a baseline for emergent behavior, but their reliance on reward-shaped agent dynamics [11, 2], which makes them unsuitable for studying unconstrained decision-making processes.

This chapter presents an overview of the current state of research related to large language models used as autonomous agents, with a particular focus on their integration within simulated environments. The purpose of this section is to analyze existing approaches, identify common design patterns, and highlight the limitations that motivate the proposed study.

2.4 Large Language Models as Autonomous Agents

Knowing the development curve and history of LLMs their agent behavior comes as no surprise. Modern day transformer-based models have various abilities as reasoning, planning, following instructions when placed in an environment with structured prompts and iterative interaction loops. These properties have motivated research into treating LLMs as autonomous decision-making entities capable of interacting with external environments through structured inputs and outputs [15, 9].

Prior studies demonstrate the capabilities of LLMs to decompose objectives, even abstract ones, into steps, adapt behavior based on feedback, be it self-reflection or external and maintain contextual awareness through multiple interactions. Frameworks such as ReAct and Reflexion explicitly exploit natural-language reasoning traces to guide action selection and behavioral refinement [15, 9]. Most academic documents and studies focus on short-term tasks or objectives with narrow scopes. As a result, long-term autonomous behavior remains insufficiently examined.

[15] introduces the ReAct framework, where language models interleave reasoning steps ("thoughts") with actions taken in an environment. The model receives observations, reasons about them in natural language, and decides which action to take next. ReAct demonstrates improved decision-making in interactive environments such as question answering and simple games.

The framework is evaluated on various tasks such as answering questions, decisions in simple interactive environments, where it demonstrates interpretability and performance comparable to standard prompting techniques. ReAct allows better tracking of the decision-making process by explicitly exposing reasoning steps.

ReAct focuses on prompt-level reasoning control, whereas my study focuses on long-term autonomous behavior in a continuous simulation loop. The relation is subtle, as I can explore the difference between controlled and not controlled reasoning.

Reflexion [9] introduces a mechanism where LLM agents self-reflect on past failures using natural language feedback. Instead of gradient-based learning, the model

updates behavior via textual self-critique stored in memory. This improves task success over repeated trials.

The system demonstrates improved success rates over repeated trials on coding and reasoning tasks. Reflexion targets task improvement, while my thesis focuses on emergent behavior and goal formation, not explicit task optimization. On a first look there is no connection between the two studies, but in depth, the task optimization should come as implicit behavior for a well developed LLM.

2.5 Language Models in Simulated and Game Environments

Simulated environments and video games have long been used as testbeds for artificial intelligence research, as they provide controlled, but expressive environments [2, 10, 13]. Classical approaches rely on reinforced learning, the agent optimizes explicit reward functions through repeated interaction[11]. Recently language models are used as an alternative controller, being capable of interpreting environment descriptions and selecting actions [13].

Articles by Park et al. [6] and by Wang et al. [13] present exciting approaches, such as placing language-based agents in a virtual town, where they plan activities, interact socially, and form memories, to obtain human-like behavior, respectively, to place LLM-based agent operating autonomously in a sandbox, open-world game (Minecraft) where it explores surroundings, acquires skills and constructs a skill library which can be reused.

Despite these successes, such systems often impose predefined objectives or constrains of action space or task structure, to simplify evaluation but limit insights. Behavior under continuous decision-making loops, partial observability and open-ended interaction over extended periods of time are some of the elements lost due to the design choices mentioned earlier.

This influential paper [6] embeds LLM-driven agents into a simulated town environment where agents form memories, plan daily routines, and interact socially. Agents demonstrate emergent behaviors such as coordination, event planning, and consistent personalities.

The system succeeding in a simulated environment suggesting that LLMs can support complex, multi-step interactions in dynamic settings. This work strongly motivates my approach. However, it focuses on social simulation, while my thesis examines autonomous decision-making and interaction mechanics in a game environment without social constraints to shape the agent's dynamic.

Voyager [13] presents an LLM-based agent operating in a sandbox, open-world game, Minecraft. The agent autonomously explores, acquires skills, and builds a skill library without human intervention. It uses a language model for high-level planning and code generation, enabling it to execute actions through dynamically

generated programs.

The agent explores the environment, collects resources, and develops increasingly complex capabilities without direct human intervention. The framework demonstrates the potential of LLMs to support open-ended learning and long-horizon interaction in complex environments. Voyager is goal-driven and skill-based. My work differs by using predefined objectives as suggestions, allowing observation of spontaneous behavior.

2.6 Tool-Using and Planning-Capable Language Models

Another prominent research direction focuses on enabling LLMs to interact with external tools, APIs or computational resources. Such models can autonomously use the tools by deciding when to invoke external functions as part of their reasoning process.

Toolformer exemplifies this approach by allowing models to learn tool usage through self-supervised signals embedded within natural language data [8].

Planning-capable systems further extend this paradigm. SayCan integrates language-model likelihoods with affordance-based feasibility constraints to select executable actions in robotic domains [1]. These approaches position the language model as a high-level planner rather than a fully autonomous agent embedded in a persistent environment.

Moreover, evaluations are performed in static or semi-static task settings, such as completing a set goal or following instructions. Consequently, the behavioral characteristics of such models in continuously evolving simulated worlds remain underexplored.

Toolformer [8] enables LLMs to autonomously decide when and how to call external tools (e.g., APIs, calculators). The model learns tool usage through self-supervised signals. This approach allows LLMs to extend their capabilities beyond static knowledge, improving performance on tasks that require precise computation or up-to-date information. The integration of tool usage into the generation process enables more flexible and accurate outputs.

Toolformer focuses on enabling language models to decide when and how to use external tools as part of their reasoning process. This extends the functional capabilities of LLMs but remains centered on task execution. In contrast, this thesis examines how an agent selects actions within an environment, emphasizing behavioral patterns rather than tool invocation efficiency.

SayCan [1] integrates LLMs with robotic planners by scoring possible actions based on language-model likelihoods and feasibility constraints. It bridges symbolic planning and real-world execution, by allowing the system to select actions that

are both meaningful and executable.

The method is applied to robotic manipulation tasks, demonstrating the ability to follow complex instructions and adapt to real-world constraints. By integrating symbolic planning with language-based reasoning, SayCan bridges the gap between abstract instructions and physical execution.

SayCan uses external planners, while my framework lets the LLM directly choose actions from contextual state representations. Comparative study of the previously mentioned approaches can lead to a series of discussions, fact used as reasoning for including this work.

2.7 Embodied and Multi-Agent AI Systems

Embodied AI research focuses on agents that perceive and act through sensory inputs and motor outputs. Such agents rely on reinforced learning and physics simulation or visual perception. In contrast, language-based agents operate on abstract representations of the environment.

Recent work combines LLMs with simulated embodiment [6, 13]. Multi-agent extensions of this approach enable agents to interact - compete or cooperate - in a shared environment [6, 2]. Studies on emergent tool use in multi-agent reinforcement learning systems demonstrate that complex coordination strategies can arise without explicit programming [2].

Recent language-based systems explore self-improving agents that iteratively refine their behavior using memory and self-generated feedback [9, 5]. However, many such systems rely on extensive scaffolding, external evaluators, or explicit improvement objectives, making it difficult to isolate and analyze the intrinsic autonomy of the language model itself.

[2] demonstrates how complex behaviors can emerge in multi-agent reinforcement learning systems through interaction and competition. Agents are trained in shared environments where they must cooperate or compete to achieve objectives, leading to the spontaneous development of strategies and tool use.

The study shows that even simple agents, when placed in sufficiently rich environments, can exhibit sophisticated coordination patterns without explicit programming. These results highlight the role of environment dynamics and reward structures in shaping emergent behavior.

This work demonstrates that complex behaviors can emerge from multi-agent interaction under reinforcement learning. The emergence observed is driven by reward optimization and interaction dynamics. In contrast, this thesis explores whether similar or different patterns arise in a language-based setting where no reward function or optimization signal is present.

Self-Refine [5] introduces a framework in which a language model iteratively improves its outputs by generating feedback on its own responses and refining them over multiple steps. Instead of relying on external supervision or gradient updates, the model uses natural language self-critique to guide improvements.

Although not designed as an autonomous agent operating in a persistent environment, the approach demonstrates how iterative feedback loops can enhance performance and adaptability across tasks.

This work relates to the thesis through its use of self-generated feedback and iterative reasoning. While Self-Refine operates in a task-based setting, it highlights mechanisms that could influence behavior over time. In contrast, this thesis investigates similar ideas in a continuous, goal-free environment, focusing on behavioral consistency rather than output optimization.

2.8 Comparative Approaches to Autonomous Decision-Making

Several paradigms address autonomous decision-making processes in complex environments, such as:

Reinforcement learning where agents optimize explicit reward functions, but lack interpretability [11].

Rule-Based Systems where agents provide predictable behavior, but are difficult to scale.

Hybrid Systems which combine abstract/symbolic reasoning with learned policies.

Language-based where agents rely on contextual understanding and reasoning.

Each paradigm involves trade-offs in terms of adaptability, explainability, and generalization. Unlike reinforcement learning agents or search-based systems such as AlphaGo, which rely on explicit optimization objectives and reward signals [11, 10], LLM-driven agents operate through implicit reasoning derived from contextual input.

This thesis situates itself within this emerging research direction by empirically examining how language models behave when granted decision-making autonomy in a simulated environment without predefined goals or reward structures.

AlphaGo [10] combines deep neural networks with Monte Carlo Tree Search to achieve superhuman performance. It represents a classical reward-optimized, closed-world AI system. The approach is trained using a combination of supervised learning from expert games and reinforcement learning through self-play, resulting in highly optimized decision-making within a well-defined environment.

AlphaGo represents a highly optimized decision-making system driven by reinforcement learning and search techniques. Its behavior is explicitly shaped by well-defined objectives and reward signals. In contrast, this thesis investigates a setting in which no such optimization criteria are provided, allowing for the analysis of decision-making driven purely by contextual reasoning.

Reinforcement learning [11] provides a formal framework for training agents through reward-based interaction with an environment. While this paradigm is highly effective for goal-oriented tasks, it inherently biases agent behavior toward optimization. The present thesis deliberately removes this component to study how an agent behaves when no reward signal is available to guide its decisions. My approach deliberately contrasts RL by removing reward shaping to observe non-optimized autonomous behavior.

2.9 Research approach

Starting from the limitations identified previously, this thesis adopts a simplified and controlled experimental approach, which focuses on isolating the autonomous behavior of a large language model from external optimization and understanding it.

The main idea of the proposed approach is to isolate capabilities from objectives. Instead of training or guiding the agent towards a goal, the LLM is provided with a fixed and complete set of actions and is allowed to operate freely within a simulated environment. The possible actions are organized in multiple levels of abstractions, including low-level primitives, such as movement or object interaction, and high-level macros that contain complex sequences of behavior.

The agent in this framework is not given notions of success, progress or goal failure, compared to prior systems which need explicit task definition or reward-driven learning [11, 13]. At each timestep, the environment state is encoded into a structured textual representation and provided as input to the language model. Based solely on this contextual information, the model selects its next action from the available skill set.

The design choice removes the common sources of bias present in existing approaches. In goal-driven systems, agent behavior is shaped by optimization pressure, making it almost impossible to determine if the observed patterns are intrinsic or induced. Similarly, external planners or scripted objectives constrain the agent's decision space [15, 1, 13]. By contrast, the proposed framework minimizes external influence, allowing the internal reasoning processes to be central in action selection.

Providing both simple actions and high-level macros is important as they ensure that the agent retains full control over its interaction with the environment and introduce the possibility of structured, multi-step behavior emerging from a single decision. The combination enables the study of how the model balances granular control with more abstract behavioral patterns over time.

Rather than evaluating performance through task completion metrics, the approach emphasizes behavioral observation and analysis. All interactions between the agent and the environment are logged, as well as the reasoning behind it, producing a detailed trace of states, actions, and transitions. This data is subsequently analyzed to identify recurring patterns, shifts in behavior, and potential signs of implicit goal formation.

Through this setup, the thesis investigates whether a language model, when equipped with sufficient capabilities but no explicit objectives, exhibits consistent, exploratory, or structured behavior. The research approach therefore shifts the focus from optimization to understanding autonomy, providing a complementary perspective to existing work on language-based agents in simulated environments.

Chapter 3

Proposed Experimental Framework

The proposed experimental research framework connects Large Language Models (LLMs) to a Minecraft survival server via `"mineflayer"`, which is a protocol library. In the framework a single Node.js process hosts an autonomous agent that observes the game state, makes decisions, and executes actions without human intervention.

In order to obtain and observe empirical behavioral nature, when survival scaffolding is provided but progression goals are not, the framework is designed configuration driven. A single JavaScript configuration file selects the decision engine, LLM provider, model, temperature, autonomous vs. guided mode, the bot connection parameters, and the experimental schedule including duration and perturbations. The same runtime supports both LLM-driven and fully deterministic heuristic agents, enabling direct comparison under identical environmental conditions.

3.1 Minecraft Server Configuration

The server runs a separate Java process. The agents communicate with it exclusively through the Mineflayer protocol layer. In Minecraft modding is possible, as it allows user-created modifications to expand or customize the base game. Modding is not added. This ensures the agent experiences Minecraft exactly as a human player would.

Server configuration details:

Version Minecraft Java Edition 1.20.4

Seed −1613247987266390429

Mode Survival

Spawn protection Disabled

Connection Local TCP

Console commands Enabled

A seed is the number or sometimes text that Minecraft's world generator uses to create the world. This number determines where mountains are, caves, villages, all the structures, and biomes, using a Linear Congruential Generator.

The mode describes the way the player interacts with the world. Five fundamental modes exist: survival, creative, adventure, spectator, and hardcore. Survival mode provides the standard experience where the player must gather resources, craft tools, manage hunger, and defend against monsters. Death results in respawning, but items are lost.

Spawn protection refers to the ability of the player to destroy blocks in the spawn area.

Console commands set to enabled allow the agents to use admin rights to modify the position, state or inventory of the bot. Commands used are `/kill @p`, `/clear @p` and `/spreadplayers 500 500 0 100 false @p`. `/kill @p` is the forced-death perturbation; because `keepInventory` is not enabled, the bot drops its items on death. `/clear @p` is the separate inventory-wipe perturbation. Teleportation uses `/spreadplayers 500 500 0 100 false @p` to place the bot on solid ground near $(500, 500)$, followed after 2 seconds by `/kill @e[type=!player,distance=..30]` to clear mobs at the destination. To understand the command a break-down is necessary: `/spreadplayers 500 500` sets the search area to coordinates $x = 500$ and $z = 500$, `0` represents the minimum distance between teleported targets, `100` shows maximum radius from the center where the player can land, `false` means that the teleportation is irrespective of teams, and`@p` selects the nearest player(agent).

3.2 Mineflayer

Version used:

- $mineflayer$@4.33.0
- $mineflayer - pathfinder$@2.4.5

It is the widely adopted bot framework for Minecraft Java Edition, as it provides full protocol-level bot control, including movement, inventory, crafting, combat and block interaction and pathfinding (mineflayer-pathfinder), which ensures that the agent can explore the terrain. Additionally the framework utilizes an event-driven architecture and zero client rendering overhead (headless-friendly), allowing the agent to operate entirely by sending and receiving network packets, without processing 3D graphics.

Framework-level caveats:

Swimming mechanics Modern Minecraft (1.13+) changed swimming physics. Early iterations did not account for this, causing drowning deaths that required patching at the action primitive level.

Pathfinder limitations `mineflayer-pathfinder` can fail on complex terrain (waterfalls, dense forests, overhangs). The framework wraps these failures

with behavior guards (`escape_trap`, `forced_recovery`) rather than modifying the pathfinder itself.

Block placement underwater Requires precise face-vector calculation in Mineflayer. This was added as an emergency drowning fallback.

Death/respawn state Long-term memory (known locations, achievements, death context) is preserved across deaths via `MemoryManager.exportState()` and `importState()`. Only short-term state such as `recentActions`, `stuckLoopCount`, and `blockedActions` is reset on respawn.

3.3 Architecture and modules

The system architecture is built around a lightweight Node.js runtime that connects a Minecraft agent to an external Large Language Model through direct HTTPS API calls. The project uses Node.js 18+ with the CommonJS module system, meaning all files rely on require and module.exports instead of ES modules. No bundler or SDK layer is used. Instead, the architecture deliberately uses Node.js' native https module to communicate with LLM providers. This keeps the implementation simple, transparent, and dependency-light, while also avoiding runtime issues caused by unavailable packages such as openai or @anthropic-ai/sdk.

At the environment level, the agent is implemented using mineflayer, which provides the Minecraft bot interface and exposes the bot's perception and action capabilities. Movement and navigation are handled through mineflayer-pathfinder, which supplies A* pathfinding for moving through the world, while vec3 is used for 3D coordinate and vector operations. The overall architecture can therefore be understood as three connected layers: the Minecraft interaction layer, which gathers world state and executes actions; the reasoning layer, where prompts are sent to the LLM through raw HTTPS requests; and the control layer, which translates model outputs into valid bot actions such as moving, mining, crafting, or interacting with blocks and entities.

Modules used and their responsibilities:

run_experiment.js (Entry point) Creates the Mineflayer bot, wires subsystems, handles reconnection across deaths, restores MemoryManager state;

agent.js (AgentRuntime) Contains the decision loop and all cross-loop states: guards, goal tracking, blocked-actions map, stuck-loop detection, interrupt flags, shelter tracking and cooldowns;

actions.js (ActionSystem) Contains over 50 action primitives for movement, crafting, combat, resource gathering. Exposes `executeAction(name,params)` with results of success/error;

observation.js (ObservationSystem) World-state extraction, which returns a structured snapshot each loop: health, food, oxygen, position, inventory, nearby entities ($\approx$ 16m radius), nearby blocks/resources, environment (time, biome, weather, terrain scan, vertical profile).

llm.js (LLMInterface) has function that serve as prompt builder, priority alert generator, raw HTTPS caller (OpenAI/Anthropic/local), JSON response parser.

heuristic_engine.js (HeuristicEngine) implements a deterministic rule-based decision engine. Implements the same `generateDecision()` interface as LLMInterface. Zero API cost. It is used as a non-LLM baseline.

logging.js (LoggingSystem) provides JSONL output, buffered writes, death rotation, run summary. Writes observations, actions, goals, LLM requests, events, memory summaries per run.

supervisor.js (ExperimentSupervisor) focuses on running the timer, perturbation scheduling, stalling watchdog (>60s loop = warning), clean shutdown on signal.

goal_manager.js (GoalManager) is the progression tier inference, doing milestone checklist and thrash detection.

memory.js (MemoryManager) stores known locations (crafting tables, beds, furnaces), achievements, death context, periodic summaries. Survives death via methods `exportState()` / `importState()`.

3.4 Decision loop

The decision loop is accompanied by two parallel safety systems: guard checks and interrupt flags, present at the start, respectively mid-action. Interrupt flag events fire during long-running block-break actions and abort them immediately. This adds sub-loop latency because a 5-second interval is too slow for drowning (≈10 seconds to death) or skeleton combat.

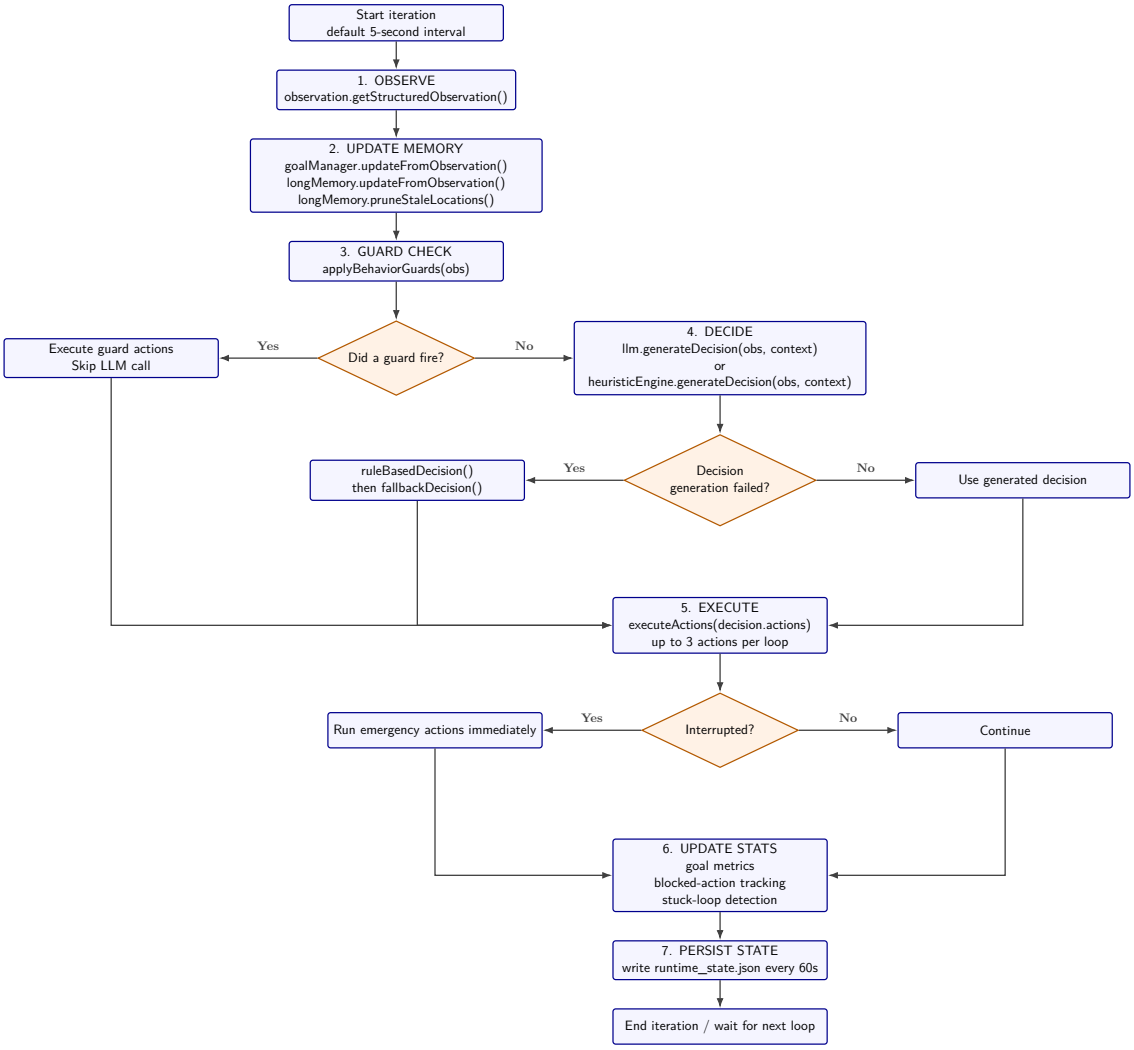

Figure 3.1: Decision loop

Guard checks function by reading the observed state and trigger if the agent is detected in a drowning/combat/stuck state. The aforementioned consist of hardcoded overrides that execute prior to the decision engine, either LLM or heuristic, is consulted. Such precautions are taken to prevent degenerate loops in which the action chosen by the model would lead to certain death or to an infinite loop. Guards are categorized into survival, anti-stuck, progression and tool durability. The survival category consists of checks for drowning, combat, low health, and rapid damage and is present in both the autonomous and guided modes. Anti-stuck guards check for path failures and trigger forced recovery if after 4 iterations of the loop the agent continues to be trapped, existing in both modes as well. Progression category contains checks for tool crafting, stone or higher quality material tools, bed crafting and night shelter and it is present in guided mode only. Similarly to the aforementioned tool durability guards are present only in guided mode, proactively replace or equip backup pickaxe/axe before breakage. The integration of safety precautions is based on the mode, as in autonomous mode (`research_autonomous.js`) the model is expected to handle tool crafting, shelter building, and progression entirely on its own, while in guided

mode (`guided_baseline.js`) scripted progression overrides the model when specific failure patterns are detected. Another mode of running the experiment is heuristic, being the baseline, and therefore the actions are completely scripted.

In light of the preceding it is imperative to detail the decision making mechanism. The framework supports two distinct engines, selectable via `config.llm.provider`. The `LLMInterface` is used by the LLM-based agents, in both guided and autonomous mode, in order to observe emergent behavior. Complementary `HeuristicEngine` measures the scripted competence, irrespective of behavioral patterns. To maintain consistency both engines use the same observation system, action primitives, guards and logging.

Apropos of `config.llm`, the module contains a boolean that splits the configuration for autonomous or guided modes. For `autonomousMode: true` next step directive is not included in prompt, only the mentioned safe guards are active and the model decides progression on its own. For `autonomousMode: false` next step directive injected based on current tier, all guards are active and the model is nudged toward scripted milestones. This coupling is intentional: the research question requires that the model sees only raw state, so removing the directive without also removing progression guards would still be scripted behavior.

3.5 Heuristic baseline and LLM agents design

A baseline is needed in experimental research to provide reference point for comparison. As such, the proposed framework uses the heuristic mode, a deterministic priority stack. It is a hardcoded state machine that inspects the world state and returns the highest-priority action that matches the current conditions.

The heuristic engine was designed to answer a fundamental question "What is the minimum scripted competence required to survive and progress in Minecraft survival, without any learning, memory, or adaptation?" Therefore the mecanism serves as a non-LLM baseline against which LLM behavior can be compared. If an LLM cannot outperform a ≈500-line rule-based system, that reveals a fundamental limitation in the model's ability to ground decisions in structured game state.

Advantages of the design include zero API cost, enabling extensive baseline validation without budget constraints, perfect reproducibility, identical behavior being a result of using an identical seed and configuration and known ceiling, provides a clear floor for "competent but mindless" play.

The LLM-based agent treats the language model as a general-purpose decision-maker. Instead of hardcoded priorities, the framework extracts a structured observation of the world state, packages it into a natural-language prompt, queries the model, to obtain the goal, reasoning and next one to three actions, providing the current state. Finally the JSON response is parsed and the action executed.

The design tests the abilities of the LLM to ground decisions in a structured game

state, consisting of position, inventory, nearby threats etc., maintain coherence across multiple decision loops, exhibit progression without explicit curriculum or rewards and to adapt to perturbations, such as death, teleportation, inventory wipe.

Chapter 4

Implementation details

This chapter offers insights into the implementation of the heuristic engine, behavioral guards, state persistence and death handling, logging and dataset generation, and input to the LLM.

4.1 Heuristic Engine Implementation

The mechanism is a priority stack which evaluates conditions in strict order and the first matching wins. The conditions are split into hierarchical categories in `heuristic_engine.js`:

Emergency survival which trigger events mandatory for survival, such as `swim_up`/surfacing to avoid water entrapment for oxygen <10/oxygen <18, flee for skeletons <12m with health <15, creepers <6 m trigger flee, very close hostiles (<5 m) with health <10, and eating or, if no food is available, attacking a passive mob or foraging leaves/bushes when health ≤6.

Deep-underground emergency when $y < 50$ and no pickaxe is available, the bot crafts a wooden pickaxe from inventory materials, crafts planks first if needed, or digs to the surface.

Night survival contains the conditions on the surface at night, sleep if a bed is available, if not build a shelter if at least 10 placeable blocks are available and no skeleton/creeper is within 15 m, otherwise dig an emergency shelter if not already in a hole ($y < 68$) and a shelter was not just dug.

Food security triggers eat when food <10 and hunt or forage when food is low and no food is in the inventory.

Inventory management means drop junk when only 2 empty slots remain, craft emergency and backup pickaxes and proactively craft a backup when pickaxe durability falls below 25%.

Progression by tier same as the canon tiers, ordered wood, stone, iron, and diamond phases.

`buildDecision()` contains approximately 45–50 distinct return rules. The norms assist in returning a decision object or continue, based on the fields from the

structure observation of the environment. An example of such rule is applied in an emergency underground pickaxe crafting situation. The directive can be described as the agent is underground, meaning that its y position is lower than 50 and not in water, has no pickaxe and managed to craft 3 items previously, then: if inventory contains more than three planks and more than two sticks a wooden pickaxe is crafted. Otherwise, if inventory contains more than one log a plank is crafted to further try and create a pickaxe.

Edge case and emergency handling consist of rules regarding survival and night protection. The agent has to have a shelter over the night to survive the environment that spawns hostile mobs. Spawn-night protection means that a bot that spawns at night or is caught outside at sunset without tools will dig an emergency shelter even bare-handed. To verify that the same shelter is not dug twice implementation provides a `alreadyInHole` threshold of y position to be smaller than 68. In order to restrict the agents decision to explore a dangerous area, post-escape cooldowns are established for events like `swim_up` and `escape_trap`.

4.2 Action primitives

Action primitives constitute the execution interface between the decision engine and the Minecraft server environment. Primitives, as the name suggests, are low level, atomic, callable operations exposed by the action system class `action.js`. They translate a declarative command, such as `chop_tree`, `craft`, `flee_from`, into a sequence of imperative Mineflayer API calls targeting the Minecraft protocol layer. Their purpose is to separate decision-making from motor execution, enabling the research framework to isolate emergent strategic behavior from the underlying mechanics of agent-world interaction. This execution stratum is situated immediately below the decision loop (AgentRuntime) and above the protocol abstraction (Mineflayer). Such a design enforces a strict separation of concerns: the LLM or heuristic engine reasons about what to do, while the action system determines how to accomplish it using pathfinding, inventory manipulation, block interaction, and entity combat.

The primitives are organized into five functional categories, mirroring the canonical skill hierarchies of the game. Below are some relevant examples:

Category	Representative Primitives	Purpose
Sensing	`get_status`, `get_inventory`, `scan_blocks`, `scan_entities`, `get_nearby_summary`	Expose world-state affordances to the observation layer without mutating the environment.
Locomotion	`go_to_near`, `explore`, `follow`, `swim_up`, `dig_to_surface`	Displace the bot through 3D space using `mineflayer-pathfinder` (A* with custom `Movements` configuration), with hazard-aware fallback logic.
Manipulation	`break_block`, `place_block`, `equip`, `open_container`	Mutate block states and inventory configurations, including best-tool selection and face-vector calculation for placement.
Production	`craft`, `smelt`, `mine`, `chop_tree`, `collect_food`	Transform raw resources into tools, fuel, or refined materials; includes recipe caching and automatic wool-color detection for bed crafting.
Survival	`eat`, `sleep_if_possible`, `flee_from`, `attack`, `build_shelter`, `dig_emergency_shelter`, `light_area`	Preserve bot viability against environmental threats (drowning, hostile mobs, night-time spawning).

Over the course of framework development critical problems arose, therefore fixes were needed. First observed issue was accidental drowning, as the agent was not swimming up fast enough. The correction of this behavior consists of three phases: movement (look up, forward, jump) for four seconds, scanning the surface and finding possible emergency air-pockets. Another identified problem was digging a shelter too deep without a pickaxe and getting trapped. The fix caps the pit at 2 blocks when no pickaxe is available and at 3 blocks when a pickaxe is available. The entrance is sealed whenever any seal material (dirt, cobblestone, stone, gravel, or sand) is available; a pickaxe is not required for sealing, because dirt/cobble can be broken by hand in about one second when the bot needs to exit. A generally existing issue was the lack of interruptions for in progress actions, for example, agent drowned or was slain while breaking blocks. The fix applied consists of checking the function `shouldInterrupt()` after each broken block, respectively abort on flags `entityHunt` (combat initiated) or `breath` (when underwater). Collectively, these patches illustrate a central commitment of the framework: action primitives must be hardened to the point where their failure rate is low enough that observed behavioral variance can be attributed to the decision engine, not to execution unreliability. Without the drowning fix, the heuristic baseline would have been unviable as a comparison standard; without the interruption system, the 5-second loop interval would have introduced an irreducible mortality bias. The patched primitives thus consti-

tute the experimental floor - the minimum executable competence required before questions of strategic emergence or LLM superiority become empirically tractable.

4.3 Behavior guards

Another major detail of the framework implementation is represented by behavior guards. They constitute a hardcoded, priority-ordered safety layer embedded within the decision loop. Their function is to intercept degenerate paths, situations in which the chosen decision engine (LLM or heuristic) emits an action sequence that would result in certain death, infinite no-operation cycles or irreversible entrapment, if executed unmodified.

`applyBehaviorGuards()` evaluates a strictly ordered sequence of guard blocks. The always-active blocks are:

1. **Drowning** triggered when `isInWater === true` and `oxygenLevel <= 5` and falls back to `dig_to_surface` after two recent `swim_up` failures.
2. **Combat danger** triggers on any hostile mob within 16 m or when the agent was recently hit. Hostiles <5 m away cause immediate eat-and-flee, while hostiles <8 m away or recent hits with health ≤ 16 trigger eat-and-flee.
3. **Starvation/health safety** triggers when `health < 12` or `food < 10` and the agent eats if food is available, otherwise attacks a nearby passive mob.

The following blocks are active whenever `config.llm.autonomousMode` is not `true` (in guided and heuristic mode):

4. Extra eating and emergency hunting guards.
5. Hunt opportunity when food or wool is needed and animals are <10 m away.
6. Mine-stone guard: has pickaxe, at least 3 logs, fewer than 8 cobblestone, and no stone pickaxe.
7. Night-cycle guards: sleep, build shelter, dig emergency shelter, and wait-in-tree handling.
8. Crafting-progression guards for wooden/stone pickaxes, stone tools, and the wood phase.
9. Bed crafting when wool and planks are available.
10. Anti-repetition guard after 4 consecutive identical goals.

Finally, always evaluated:

11. **Stuck/deadlock** indicated by `stuckLoopCount`, where a value larger than 4 triggers forced recovery.
12. **Escape trap** triggers on repeated path failures, confinement, or being stranded high up.

13. **Tool durability** triggers if pickaxe or axe held has durability <15%, the agent equips a backup or crafts a replacement. This guard is active in all modes.

There is no separate **skeleton-flee** guard in `applyBehaviorGuards()`; skeletons are handled by the generic combat-danger guard. The heuristic engine contains its own skeleton-flee rule in `heuristic_engine.js`.

A parallel safety system operates below the guard layer. The Mineflayer events used are `entityHurt` and `breath`. `entityHurt` interrupts when health ≤12, two or more hits occurred within 5 seconds, rapid damage (3+ hits in 5 seconds) is detected, or the attacker is a ranged mob. `breath` interrupts when oxygen ≤5 and `isInWater === true`. `Actions.shouldInterrupt()` is checked inside `breakBlock`, `chopTree`, and `mine` after each block break. The class `Actions` exposes `shouldInterrupt()` as a callback to `AgentRuntime`. The function `shouldInterrupt()` is checked for production primitives `breakBlock, chopTree, mine` after each block break. If the flag is set, meaning that the interruption happens, the action returns `{interrupted: true}` and the action loop aborts remaining queued actions. Next the `executeDecisionStep()` dispatches an emergency action sequence (`eat, swim_up, or flee_from`) before yielding to the next loop. This mechanism closes the temporal safety gap between the 5-second observation cadence and the ≈2–10 second time-to-death for drowning or skeleton barrages.

Having discussed the safety systems, an inquiry into how guards and decisions interact is mandatory. When a guard is triggered, the decision object fully bypasses the LLM. Method `FilterBlockedActions()` ensures that actions imposed by the guards respect the restrictions on recently failed primitives, while preserving `neverBlock()` whitelist for survival actions. Post-execution, `trackActionCooldowns()` updates `lastDigShelterLoop()` and `blockedActions` to prevent immediate recurrence.

4.4 Logging

A major detail of the implementation is the logging subsystem, which constitutes the empirical instrumentation layer. It is responsible for registering the execution states of the experiments into persistent, structured datasets suitable for quantitative behavioral analysis. Because the research design relies on after-the-fact comparison of decision-engine trajectories, across survival metrics, progression, and emergent strategy patterns, the integrity, completeness, and schema uniformity of the generated logs are as critical as the correctness of the agent logic itself. The class `LoggingSystem`, contained in `logging.js` implements a multi-stream, buffered output architecture that records every observation, action, decision, LLM request, and critical event as timestamped JSON Lines (JSONL), enabling efficient append-only writes.

Each experiment generates a unique directory under `mc_agent/runs/`, named in the format `{runName}_{HH-MM-SS}/`. Inside the folder, life subfolders are created,

to preserve information even for a run with multiple deaths. This structure reflects the experimental reality that a single 60-minute run may contain multiple deaths and respawns, each representing a discontinuous segment of agent existence with reset short-term state but preserved long-term memory.

The complete file and folder hierarchy is:

```
runs/{experimentName}_{HH-MM-SS}/
|-- life_01/
|   |-- observations.jsonl
|   |-- actions.jsonl
|   |-- goals.jsonl
|   |-- llm_requests.jsonl
|   |-- events.jsonl
|   |-- memory_summaries.jsonl
|   |-- metadata.json
|   |-- summary.json
|   |-- behavioral_analysis.json
|-- life_02/
|   |-- [idem]
|-- experiment_summary.json
```

Each of the files generated under the life subfolders respond to specific research questions and serve a distinct analytical function. An in-depth analysis would be:

`observations.jsonl` — World State per Loop
Records the complete structured observation emitted by `ObservationSystem.getStructuredObservation()` at the start of every decision step. Each line contains the player state, including position, health, food, and oxygen, together with the inventory summary, nearby entity lists, block resources, environmental data such as time, weather, light level, and terrain scan, and the tick timestamp. This file is the primary input for survival-curve analysis, position-trajectory reconstruction, and environmental context modeling.

`actions.jsonl` — Executed Actions with Results
Logs every action dispatched by `executeActions()`. Each entry includes the action name, parameter payload, success status, execution duration, error message when applicable, and primitive-specific metadata, such as `blocksPlaced`, `distanceTraveled`, or `rose` for vertical displacement. This stream is used to compute action success rates, identify failure-prone primitives, and detect blocked-action patterns.

`goals.jsonl` — Decisions and Guard Overrides
Captures the high-level decision object returned by the LLM, the heuristic engine, or a behaviour guard. Guard-fired decisions are annotated with `isGuard: true`, which makes it possible to distinguish between the agent's own decision process and external safety interventions. The recorded fields include the declared goal, reasoning string, action array, and loop-level context. This file supports goal-coherence scoring, thrash detection, and guard-intervention frequency analysis.

`llm_requests.jsonl` — Raw Prompt-Response Pairs
Stores the complete LLM interaction cycle, including the request ID, provider and model, full prompt text, raw response content, parsed decision, token usage, latency in milliseconds, and parse errors. Keeping this stream separate from `events.jsonl` avoids overloading the critical-event log with high-volume text data. This file is the main resource for prompt-engineering post-analysis, token-budget auditing, and failure-mode inspection.

`events.jsonl` — Critical Event Log
Provides an append-only record of discrete events that change the experimental state, such as `BOT_DEATH`, `BOT_SPAWN`, `PERTURBATION`, `WATCHDOG_STALL`, `GOAL_THRASH`, `ACTION_BLOCKED`, `DANGER_RESPONSE`, `LOOP_ERROR`, and `EXPERIMENT_END`. The transition from an earlier read-modify-write `events.json` array to append-only JSONL removed an unbounded I/O bottleneck during long-running experiments.

`memory_summaries.jsonl` — Periodic Cognitive Snapshots
`MemoryManager.buildRecentSummary()` is rebuilt every 10 loops and the resulting summary is included in each LLM prompt. The harness does not currently call `logger.logMemorySummary()`, so this file is not persisted to disk. These entries include recent position trajectories, inventory changes, achievement lists, known locations, and death context. The file acts as a periodic approximation of the agent's working memory and supports analysis of long-horizon location recall and spatial reasoning.

`metadata.json` — Run Configuration Snapshot
Is a static JSON file written during initialization. It records the experiment name, Minecraft seed, LLM configuration with API keys redacted, perturbation schedule, and action-set enumeration. This file serves as the provenance record for each dataset.

`runtime_state.json` — Persisted Cross-Loop State
Is overwritten every 60 seconds and contains serialized snapshots of the `GoalManager` and `MemoryManager` state. Its purpose is to support experiment resumption after process crashes and to preserve achievement history across deaths.

`summary.json` — Per-Life Aggregate Statistics
Is generated during log finalization, through `close()`. It contains the total duration, line counts for each JSONL stream, and embedded metadata. This file provides a lightweight index for dataset cataloging.

`behavioral_analysis.json` — Computed Behavioral Metrics
Is a post-processed analytics file generated at the end of each life. It pre-computes metrics such as decision coherence rate, action-type frequency distribution, most frequent goals, maximum consecutive failures, and perturbation recovery latency. This file accelerates comparative analysis across runs by avoiding repeated scans of the raw JSONL streams.

4.5 Input to the LLM

Whilst guards override the behavior, prompts offer the LLM the ability to perceive, evaluate and reason about the environment and make a decision. The prompts constitute the only information channel through which the LLM receives structured observations. The experimental framework adopts a symbolic grounding paradigm, meaning structured observation extracted by `ObservationSystem` is serialized into natural language, packaged into a context window, and transmitted to the LLM as a text completion task. The design of the prompt is a control variable, as a consequence of section ordering, information density, semantics and vocabulary. The phrasing, alert priority and inventory depletion are considered perturbations as they can shift the behavior of the LLM in possibly unexpected ways. Therefore, prompt construction is a vital detail of implementation to offer consistent and coherent behavior.

The construction is done in `llm.js` class, in the function `buildTurnPrompt()`, following a rigid template that mirrors the cognitive requirements of survival game play: perceiving environment (state), evaluating strategic position, recalling historical context (events), considering warnings/danger and choosing a response. The cognitive requirements are mapped to sections of the prompt, that get concatenated.

State division contains:

```
Position: (x, y, z) [Y: surface / underground (ores Y{n}) / bedrock]
Standing on: {blockName}
Health/Food/Oxygen: {h}/20, {f}/20, {o}/20
Time: {ticks} ({dawn|day|noon|afternoon|dusk|night})
Weather: {rain|clear}
Light level: {effective}/15 {MOBS CAN SPAWN HERE | (safe)}
Held tool durability: {item} {pct}% ({remaining}/{max})
Crafting table: {in inventory | placed nearby {d}m | none}
Empty slots: {n} {(FULL - do not gather more; craft or explore)}
Inventory: {item}x{count}, ...
Nearby mobs: {name}@{distance}m, ... | none
Nearby resources: {name}@{distance}m, ... | none
```

The Y-coordinate labels the zone, in addition to the value itself. Such implementation reduces the cognitive load of interpreting elevation. The light level includes a binary `mobSpawnRisk` flag computed from the effective light threshold (< 8 in Minecraft 1.20.4), translating a continuous environmental variable into a discrete threat predicate. Tool durability is both expressed as percentage and number of uses remaining, enabling the model to anticipate breakage before it occurs. `CompactMode` is responsible for inventory truncation, by showing only the top 8 items (versus 14 in standard mode) and only 4 mobs (versus 6), when cumulative token usage exceeds 85%.

Strategy section of the prompt situates the agent within the progression hierarchy and contains:

```
Current tier: {naked | wood_tools | stone_tools | iron_tools
```

```
| shelter | established}
Next milestone: {milestone}
Missing for milestone: {item1, item2, ...}
[guided only] NEXT STEP: tier-based directive
```

In guided mode the "NEXT STEP" directive is appended, for example at the naked tier with insufficient logs, the directive reads: `"Chop trees for wood: { "name": "chop_tree", "params": { "count": 3 } }"`. In autonomous mode the directive is omitted, leaving the agent to infer progression sequencing from state. The design choice allows emergent strategic planning to be visible, compared to scripted scaffolding. In addition, a post-escape cooldown note is injected if the model recently executed `escape_trap` or `forced_recovery`.

Memory part of the prompt contains a recent summary including position, inventory changes, average mob presence over last 10 loops, described in natural language and generated by `MemoryManager.buildRecentSummary()`. Another line of memory input contains achievements that the game provides at certain progression steps, offering the model a coarse-grained autobiographical record. The next entries refer to known locations for crafting tables, beds, furnaces and death sites. Related to last death there is another line that consists of a compact fatality report, as `Died at (x, y, z) from {cause}. Was drowning.` to inform hazard avoidance. This section addresses the problem of memory persistence in the inner-loop, because the model has no internal state across API calls, therefore all historical context must be sent explicitly in each prompt.

Due to the persistency issue, the input to the LLM contains the previous strategy as well, structured in pairs of goal and reasoning. The way of working encourages consistency by showing the model its own prior justifications, the prompt reduces goal thrashing and reinforces multi-step plan continuity.

Priority alerts are also included in the prompt. A maximum of two alerts are are selected, based on the ranking: `drowning > underground > trapped > path_fail > stuck > hostile_mob > low_health > night > food > inventory_full`. Each alert is a natural-language string, to prevent prompt bloating and preserving the most urgent environmental signals.

The closing instructions differ by the experimental run. For autonomous the termination of the prompt includes "Choose 1–3 concrete actions based on your own reasoning. Respond with valid JSON only", while the guided one has "Choose 1–3 concrete actions. Follow the NEXT STEP unless an alert overrides it. Respond with valid JSON only."

The tables contain an example of metadata received from the LLM.
Below is a complete example with a prompt used in one of the runs for the guided run:

```
STATE
- Position: (550.5, 64.0, 420.5) [Y: surface]
```

Field	Value
Provider	OpenAI
Model	`gpt-4o-mini`
Prompt length	2523 characters
Tokens used	1386
Duration	3080 ms
Success	`true`
Error	`null`

```
- Standing on: grass_block
- Health/Food/Oxygen: 14/20, 6/20, 1/20
- Time: 14065 (dusk)
- Weather: rain
- Light level: 0/15 [MOBS CAN SPAWN HERE]
- Held tool durability: stone_axe 94% (123/131)
- Crafting table: in inventory
- Empty slots: 36
- Inventory: oak_logx1, wooden_pickaxex1, stickx10,
  oak_saplingx1, dirtx3, stone_pickaxex1, birch_saplingx1,
  eggx3, stone_axex1, crafting_tablex1
- Nearby mobs: none
- Nearby resources: You are mostly exposed to open sky.
  Hazards: cliff_or_hole 1 blocks north; cliff_or_hole 1 blocks
  ↪ east;
  cliff_or_hole 1 blocks south east.
  Nearby resources: sand 1m, dirt 2m, water 3m, stone 5m, gravel
  ↪ 13m.

STRATEGY
- Current tier: stone_tools
- Next milestone: bed_and_shelter
- Missing for milestone: wool (kill sheep), planks, bed, furnace,
  coal, iron_ore, iron_ingot

NEXT STEP (do this NOW unless a PRIORITY_ALERT overrides):
DIG EMERGENCY SHELTER - it is night!
{ "name": "dig_emergency_shelter", "params": {} }

MEMORY
- Recent summary: Last 10 loops: moved from (73, 83, -7)
  to (111, 68, -15); inventory delta -18; avg nearby mobs 0.
- Achievements: crafted_wooden_pickaxe, reached_stone_tier,
↪ crafted_furnace
- Known locations: spruce_log@(166,70,-194), dirt@(550,62,420),
  gravel@(556,53,416), sand@(550,63,421), water@(553,62,420)
- Last death context: Died at (166.04, 70, -195.9) from unknown.

PREVIOUS STRATEGY (your last 2 goals)
```

```
- Goal: "build shelter"
  Reasoning: "It is night and I need to create a safe space to avoid
   ↪  mobs."
- Goal: "gather resources for shelter"
  Reasoning: "I need to gather materials quickly to build a proper
   ↪  shelter
  before mobs spawn."

RECENT ACTIONS (last 5)
- dig_emergency_shelter: OK
- chop_tree: OK
- craft: FAILED -- Cannot read properties of undefined (reading
↪  'replace')
- chop_tree: OK
- mine: FAILED -- No block type specified. Use mine with blockType
↪  like
  "stone", "coal_ore", "iron_ore", etc.

PRIORITY_ALERTS
1. Night on surface. Build shelter or dig emergency shelter.
2. Low food and no food inventory. Hunt animals now.

BLOCKED (do NOT use these this turn):
build_shelter, dig_emergency_shelter

AVAILABLE ACTIONS
chop_tree, mine, craft, attack, eat, explore, flee_from, swim_up,
dig_to_surface, build_shelter, dig_emergency_shelter,
↪  sleep_if_possible,
pillar_up, break_around, light_area, go_to_near, equip, drop_item,
collect_food

Choose 1-3 concrete actions. Follow the NEXT STEP unless an alert
↪  overrides it.
Respond with valid JSON only.
```

4.6 Output from LLM

The output contains the goal the LLM has chosen, described in natural language, the reasoning behind this decision and actions. The last is an array of one to three actions name, which must match an existing action. Goal and reasoning are logged for interpretability and do not affect execution directly. On parse failure or invalid structure the framework has a fallback. The schema is:

```
{
  "goal": "string - current goal description",
  "reasoning": "string - explanation",
  "actions": [
    { "name": "string", "params": {} }
```

```
  ]
}
```

An example of complete response from LLM is:

```
{
  "goal": "build shelter",
  "reasoning": "It is night and I need to create a safe space to
   ↪  avoid
  mobs, but I cannot dig an emergency shelter again. I will try to
   ↪  gather resources for a proper shelter instead.",
  "actions": [
    {
      "name": "chop_tree",
      "params": {}
    },
    {
      "name": "craft",
      "params": {}
    },
    {
      "name": "mine",
      "params": {
        "blockType": "dirt"
      }
    }
  ]
}
```

The implementation described constitutes the executable substrate of the experimental framework. Each subsystem, heuristic engine, action primitives, behavior guards, state persistence layer, logging infrastructure, LLM prompt architecture has been iteratively hardened against the failure modes characteristic of embodied Minecraft agents. These hardening steps were not merely engineering polish, they were necessary preconditions for the validity of the experiments that follow. A framework in which the action layer fails unpredictably cannot produce meaningful comparisons between decision engines, because observed performance would conflate cognitive capability with execution unreliability. By unifying all configurations around a common observation schema, a shared action vocabulary, and a consistent logging format, the implementation ensures that the only systematic variable manipulated across runs is the decision engine itself. With the technical foundation now established, next chapter turns to the experimental design.

Chapter 5

Experimental design

The experiment is designed to research whether an LLM can exhibit strategic behavior in an open-ended environment without explicit objectives, under what conditions does an LLM outperform or underperform a hardcoded deterministic baseline, how much impact do behavioral guards have on improving performance of a weaker LLM and what behavior is lost through over-scaffolding and and, last but not least, how sensitive is emergent behavior to reliability of action primitives. By "emergent," we mean behavior that is not explicitly encoded in the prompt or action primitives but arises from the model's capacity to maintain coherence across multiple decision loops, anticipate future needs (e.g., gathering wood before sunset), and re-plan after disruptions. This comparative approach frames the heuristic engine not as a strawman but as a competence floor—a known, reproducible standard against which the LLM's generalization can be measured.

If the LLM cannot exceed the heuristic baseline in dimensions such as progression speed or inventory efficiency, that would constitute evidence for a fundamental grounding or planning limitation. Furthermore, autonomous-versus-guided split as an experimental treatment. By comparing GPT-4o-mini with full guards (guided) against the same model with safety-only guards (autonomous), we can isolate the marginal contribution of scripted progression scaffolding. We also hypothesize that excessive scaffolding may suppress spontaneous strategic exploration, producing competent but rigid behavioral traces.The last direction of research is investigated by holding the action layer constant across all configurations and measuring whether LLM-driven agents exhibit qualitatively different failure-recovery patterns compared to the heuristic baseline when primitives fail.

The experiment uses a between-configurations design with four distinct run types, as below:

Engine	Model	Mode	Dur.	Guards	Purpose
Heuristic	—	—	60 min	Survival, anti-stuck, progression	Scripted competence baseline
LLM	Claude Sonnet 4.6	Autonomous	60 min	Survival, anti-stuck	Strong autonomous behavior
LLM	GPT-4o-mini	Guided	60 min	Survival, progression, anti-stuck	Scaffolding impact

The heuristic baseline serves as a scripted control. It represents the minimum competence required to survive and progress. Its nature permits unlimited runs at zero API cost, reason for it being used to tune the action layer and validate the stability of the environment.

The upper-bound treatment of the runs is represented by Claude Sonnet in autonomous mode. This configuration tests a currently decently capable reasoning model and its ability to formulate and follow strategies, by having only safety guards and no next step directive. The model was chosen at project inception for its reputation.

The weak-model treatment is covered by the run of GPT-4o-mini, in the guided mode. The model was chosen because it is widely available, cheaper and considered weaker than Claude. The guided mode is used to test whether deterministic guards can compensate for inferior model capacity.

5.1 Controlled variables

To ensure that performance differences are determined by the reasoning and decision of the model the following variables are constant across all configurations:

World seed with value of -1613247987266390429, which guarantees identical terrain, resource generation, and distribution.

Server configuration with Minecraft Java Edition 1.20.4, missing online mode, no spawn protection and normal difficulty, further described in described in section 3.1.

Observation system instances are identical across all runs, with scalar normalizations for oxygen, entity radii and terrain scan depth.

Action primitives stored in modules used across configurations, including the pathfinder configuration, recipe cache, swim up three-phase protocol etc.

Perturbation schedule being a fixed timing and type sequence(teleportation/sudden death/inventory wiping) within each run. They can be found under `ExperimentSupervisor`.

Loop interval has a constant value of 5000 milliseconds, enforcing identical decision latency across all engine types.

Token budget The LLM configs set a per-request `maxTokens` limit (600 for `research_autonomous.js`, 500 for `guided_baseline.js`) and a run-level `tokenBudget` (2 million for `research_autonomous.js`, 1 million for `guided_baseline.js`). The heuristic baseline makes no LLM calls and therefore has no token budget. Compact mode is enabled when cumulative token usage exceeds 85% of `tokenBudget`, and disabled again when it drops below 60%.

Initial condition On first spawn the time of the day is set to mid-morning to eliminate night-spawn mortality bias.

5.2 Metrics and Evaluation

To move from descriptive observation to comparative inference, the experimental framework requires a structured evaluation system capable of quantifying performance across multiple behavioral dimensions. The metrics presented in this section are designed to operationalize the research questions, transforming raw execution traces—observations, actions, goals, and events—into interpretable indicators of survival competence, progression efficiency, and strategic coherence. Rather than relying on a single scalar success criterion, the evaluation adopts a multi-faceted profile that distinguishes between short-term viability (survival time), mid-term capability (tool tier advancement), long-term strategic quality (goal coherence and exploration). All metrics are derived automatically from the JSONL datasets generated by the logging subsystem, ensuring that the evaluation pipeline is reproducible, auditable, and decoupled from the subjective interpretation of individual runs.

The evaluation framework is organized into five metrics categories, providing extensive data for each configuration.

Survival metrics refer to survival time and death details. The longevity is recorded in milliseconds for maximized precision, until death or the timer for the run ends. The experiments include timers of different lengths. Furthermore, the number of deaths of the LLM along the run is recorded, as well as the cause.

Progression metrics regard the tiers reached, respectively milestones. The highest rung of the ordered hierarchy `naked < wood_tools < stone_tools < iron_tools < shelter < established` represents the maximum tier reached by the LLM, as inferred by `GoalManager` from inventory snapshots. Another metric regarding tiers depends on the time, as it registers the minutes elapsed from spawn until each tier threshold is reached. To complete the progression metric concrete milestones are recorded through boolean

indicators for various game predefined progression points.

Efficiency benchmarking is achieved through noting action rates per minute or success status, goal and redundant loop observation. Successfully executed actions are divided by survival time to register motor throughput, while the blocked actions are counted. The rate of blocked actions determines the portion of actions emitted by the decision engine that are vetoed by behavioral guards or filtered by the `blockedAction` map. Efficiency, in the context of LLM behavior includes goal formation or thrashing, therefore the framework includes a count of unique declared goals, within a sliding 5-loop window. High thrashing indicates strategic incoherence. To complete the metric, the number of duplicate crafting events for identical item class is kept.

Paramount benchmarking for this thesis is emergent behavior, focusing on goals, exploration and adaptation. Coherence of the goal is measured through a score, computed as a modal frequency within non-overlapping 3-minute window. The metric targets temporal consistency of the goals declared by the LLM where a higher score represents a sustained strategic commitment. Finally, and importantly, the ability of the LLM to adapt is measured, by storing the time between a perturbation event and the agent's return to a coherent, goal that is not survival oriented.

Chapter 6

Results and analysis

The empirical evaluation focuses on the three Minecraft agent configurations: heuristic baseline, the guided LLM baseline, and the autonomous LLM configuration. The goal is to understand both how well each agent survives and how coherently it pursues goals, recovers from setbacks and progresses, when given survival scaffolding, but no explicit instructions.

Comparable experimental environment over runs is guaranteed by using one-hour sessions on fresh identical worlds, each interrupted by the same three perturbations: a forced death, a sudden teleport to unknown terrain, and a complete inventory wipe. For comparison purposes, from these runs information is extracted and categorized as detailed metrics (survival/progression/behavioral).

The results show a clear trade-off. The heuristic agent is the most stable survivor but the least flexible: it follows fixed rules and never advances beyond the early game. The guided LLM reaches slightly further but is brittle, often falling into death spirals after a setback. The autonomous Claude agent produces the most diverse, context-sensitive behavior and reaches the highest progression ceiling, yet it still struggles with the same mechanical failures—escaping caves, swimming, and finding food—that limit all three configurations. These findings suggest that better low-level action execution, rather than simply a stronger language model, may be the key to breaking through the current mid-game ceiling.

6.1 Heuristic baseline

The data extracted for testing the heuristic baseline was obtained across 5 independent runs in fresh but identical worlds, lasting 60 minutes each. Fresh world means an environment where the changes of the previous experiment are not present. Another detail that influences the metrics are scheduled perturbations (death after 10 minutes, teleport after 20 minutes, inventory wipe after 30 minutes).

Below are illustrated the results of the experiment:

Table 6.1: Heuristic baseline: aggregate survival data

Run	Lives	Deaths	Mean life (min)	Final life (min)
1/5	4	3	14.8	34
2/5	3	2	19.8	39
3/5	4	3	14.7	3
4/5	3	2	19.6	36
5/5	4	3	14.7	25
Mean	**3.6**	**2.6**	**16.4**	**27.4**
Range	**3–4**	**2–3**	**14.7–19.8**	**3–39**

Survival data shows that across 18 lives, there are 13 deaths and 5 survivors.

Table 6.2: Heuristic baseline: progression reach

Milestone	Lives reaching it	% of lives
Stone pickaxe	7 / 18	39%
Stone axe	7 / 18	39%
Furnace	5 / 18	28%
Iron tools	0 / 18	0%
Bed	0 / 18	0%
Food item in final inventory	3 / 18	17%

The heuristic engine reaches stone tools in roughly 2 in 5 lives and a furnace in just over 1 in 4. It never reaches iron tools or beds, and it rarely carries food at life end.

Table 6.3: Heuristic baseline: action reliability

Action	Success / Total	Success rate
`wait`	125 / 125	100%
`move_to`	12 / 12	100%
`build_shelter`	11 / 11	100%
`chop_tree`	100 / 116	86%
`dig_emergency_shelter`	9 / 11	82%
`explore`	99 / 138	72%
`break_around`	46 / 65	71%
`mine`	39 / 60	65%
`craft`	95 / 169	56%
`attack`	13 / 29	45%
`dig_to_surface`	3 / 10	30%
`swim_up`	4 / 18	22%
`pillar_up`	5 / 33	15%

Basic actions (`chop_tree`, `explore`, `mine`) work well. Escape actions (`pillar_up`, `swim_up`, `dig_to_surface`) and food acquisition (`attack`) fail frequently, which drives many deaths.

Table 6.4: Heuristic baseline: goal distribution (aggregate)

Goal	Count	Interpretation
`gather_wood`	247	Dominant early-game loop; engine defaults to wood when uncertain
`progress_from_logs`	131	Converting logs to planks/sticks/crafting table
`wait_in_shelter`	125	Long night waits; often the bot stays in shelter far longer than needed
`stock_food`	70	Food-seeking attempts, usually failing because no mobs are nearby
`mine_stone`	57	Stone acquisition attempts
`escape_trap`	49	Recovery attempts from confinement
`mine_coal`	47	Fuel/light acquisition

The goal formation is consistent with the action reliability metrics, as `escape_trap` is one of the least decided upon goals.

Table 6.5: Heuristic baseline: death causes

Cause	Count	% of deaths
Environmental / starvation / drowning	6	46%
Hostile mob	7	54%
Total deaths	**13**	**100%**

Half the deaths are environmental: the bot starves while looping on `stock_food/attack` with no passive mobs nearby, or drowns because `swim_up` and `dig_to_surface` fail. The other half are zombies killing a trapped or slow-to-react bot.

Metrics are important as long as they have interpretation. In the current experiment the baseline represents the strongest deterministic rule set the project can express without an LLM. Currently it is able to gather wood, craft materials and tools and upgrade to stone tools, eat when hungry, flee when hit, hide at night, mine and place crafting tables. It lacks the capabilities to plan around absence of passive mobs when in need for food, escape confinement when guarded actions fail. The baseline performs terribly at prioritizing after inventory wipe beyond restarting from wood and reasoning that waiting in shelter for 30+ loops is wasteful.

For an LLM driven experiment, outperforming the baseline represents having fewer deaths, suggesting better survival choices, longer final life, higher progression ceiling, such as discovering iron or diamond and building beds. Other signs of better performing agents are a diversity of goals, indicating a breakout from the `gather_wood` and `wait_in_shelter`, and faster recoveries after perturbations.

6.2 Guided LLM

This experiment has the same parameters, except the decision maker, which is GPT-4o-mini. Below are the results recorded and remarks relative to the baseline:

Table 6.6: Guided baseline: aggregate survival data

Run	Lives	Deaths	Mean life (min)	Final life (min)
1/5	5	4	11.8	2
2/5	3	2	20.0	34
3/5	4	3	15.0	23
4/5	3	2	20.0	32
5/5	7	6	8.4	4
Mean	**4.4**	**3.4**	**13.5**	**19**
Range	**3–7**	**2–6**	**8.4–20.0**	**2–34**

Across 22 lives: 17 deaths, 5 survivors. Relative to the heuristic floor (§3), the guided config died more often per run (3.4 vs 2.6) and produced shorter average lives (13.5 vs 16.4 min). However, it also reached higher progression tiers in the lives that survived long enough.

Table 6.7: Guided baseline: progression reach

Milestone	Lives reaching it	% of lives
Stone pickaxe / stone axe	10 / 22	45%
Furnace	4 / 22	18%
Bed	0 / 22	0%
Iron tools	0 / 22	0%

It is mandatory to note that progression level was highly uneven:

Table 6.8: Guided baseline: run-level progression

Run	Reached stone tools	Reached furnace
1/5	No	No
2/5	3/3 lives	1/3 lives
3/5	2/4 lives	0/4 lives
4/5	2/3 lives	0/3 lives
5/5	3/7 lives	3/7 lives

The NEXT STEP directive and progression guards push the model further, as 45% of lives reach stone tools, compared to 39% for the heuristic baseline. Reaching a furnace is available for 18% of lives, while in the heuristic baseline it is present in 28% of lives. Progression is visible, but the model is fragile, a single failed crafting table placement or disorientation after perturbations can reset growth.

Table 6.9: Guided baseline: action reliability

Action	Success / Total	Success rate
`wait`	36 / 36	100%
`build_shelter`	175 / 176	99%
`chop_tree`	148 / 161	92%
`explore`	95 / 144	66%
`break_around`	58 / 91	64%
`mine`	50 / 82	61%
`flee_from`	6 / 10	60%
`dig_emergency_shelter`	9 / 18	50%
`craft`	132 / 302	44%
`eat`	10 / 23	43%
`dig_to_surface`	4 / 18	22%
`pillar_up`	9 / 56	16%
`attack`	3 / 26	12%
`swim_up`	2 / 23	9%
`equip`	0 / 14	0%

The weak points are identical to the heuristic baseline: escape actions and food acquisition fail often. A new prominent failure is for crafting, as the agent tries to create stone tools without a crafting table nearby.

Table 6.10: Guided baseline: goal distribution (aggregate)

Goal	Count	Interpretation
`build shelter`	163	Dominant goal; the agent spends a large fraction of its time reacting to night/danger
`progress_from_logs`	93	Re-bootstrapping after death/wipe
`gather wood for tools`	79	Early-game wood loop
`escape_trap`	72	Post-teleport or post-wipe recovery attempts
`mine_stone`	64	Stone-tier progression
`progress_to_stone_tools`	61	Guard-driven crafting push
`hunt for food`	51	Food-seeking, often unsuccessful
`craft_wooden_pickaxe`	50	Repeated re-crafting after wipes
`escape_drowning`	43	Water hazard response
`wait_in_tree`	26	Night/danger avoidance
`replace_worn_tool`	21	Durability guard
`gather wood for crafting`	19	Resource prep

The distribution is more diverse than the heuristic baseline, but a large share of goals are responses to the environment (build shelter, `escape_trap`, `escape_drowning`, `replace_worn_tool`) rather than proactive progression.

Table 6.11: Guided baseline: death causes

Cause	Count	% of deaths
Hostile mobs (zombie, skeleton, spider, creeper, pillager, drowned)	12	71%
Environmental / starvation / drowning	5	29%
Total deaths	**17**	**100%**

Mob deaths dominate, especially zombies and skeletons. The guided agent frequently exits a shelter into a dangerous area with low health/food and is killed before it can re-armor or re-equip. Environmental deaths are lower than in the heuristic baseline because the guard layer handles drowning and starvation more aggressively.

Perturbation recovery in the experiment seems to be overall good. At minute 10 of the run. forced death takes place and seems that the agent manages to recover, as it gathers wood and establishes basic tools within 1 to 2 lives. Teleportation at minute 20 produces mixed results, in some runs the agent adapted to the new terrain and continued; in others it entered water or a hostile biome and died repeatedly.The last perturbation, inventory wipe, indicates the weakest point of the model. Run 5/5 shows the pattern clearly: after the wipe the agent died five more times in cascading short lives, often because it could not quickly re-craft a crafting table and stone tools, or because it kept attacking mobs while unarmed and hungry.

The results indicate that guidance improves progression ceiling, GPT-4o-mini reaches stone tools more reliably than the heuristic engine and occasionally crafts a furnace. It does not, however, reach iron or beds. Another confirmed detail is that a weak model and strong guards are not enough for stable survival, because the agent dies more often and lives shorter lives than the heuristic baseline. The guards keep it alive through many immediate threats, but the model cannot form a robust recovery plan after major setbacks. Post-wipe death spirals are common.

6.3 Autonomous LLM

This experiment takes place in autonomous mode, with minimal guards and no explicit progression directive. Below are the results:

Table 6.12: Research autonomous: aggregate survival data

Run	Lives	Deaths	Mean life (min)	Final life (min)
1/5	3	2	19.7	36
2/5	4	3	14.8	22
3/5	4	3	14.8	27
4/5	5	4	12.0	22
5/5	4	3	15.0	27
Mean	**4.0**	**3.0**	**14.8**	**26.8**
Range	**3–5**	**2–4**	**12.0–19.7**	**22–36**

The autonomous config sits between the heuristic and guided baselines on raw survival: fewer deaths per run than guided (3.0 vs 3.4) but more than heuristic (2.6). Average life duration is similar to guided (14.8 vs 13.5 min), but the final lives are consistently longer, suggesting that when Claude stabilizes it stays stable longer.

Table 6.13: Research autonomous: progression reach at run level

Milestone	Runs reaching it	% of runs
Stone pickaxe / stone axe	5 / 5	100%
Furnace	4 / 5	80%
Bed	0 / 5	0%
Iron tools / iron ingot	0 / 5	0%

Claude reaches stone tools in every run and crafts a furnace in 4 out of 5 runs, a clear progression ceiling above both the heuristic and guided configurations. It also actively mines coal and, in run 4/5, collects raw copper. However, it never reaches iron tools or beds.

Table 6.14: Research autonomous: action reliability

Action	Success / Total	Success rate
`go_to_near`	30 / 31	97%
`chop_tree`	64 / 71	90%
`equip`	41 / 49	84%
`dig_emergency_shelter`	14 / 19	74%
`break_around`	77 / 105	73%
`mine`	42 / 59	71%
`explore`	66 / 94	70%
`build_shelter`	17 / 22	77%
`craft`	127 / 201	63%
`flee_from`	5 / 8	63%
`dig_to_surface`	6 / 19	32%
`eat`	3 / 10	30%
`light_area`	6 / 22	27%
`swim_up`	4 / 19	21%
`pillar_up`	12 / 65	18%

Compared to the guided baseline, Claude's craft success rate is much higher (63% vs 44%), reflecting better reasoning about ingredients and crafting-table proximity. equip also works reliably (84% vs 0% in guided). The same action failures persist, however: `pillar_up`, `swim_up`, `dig_to_surface`, and food acquisition are weak. Notably, mine succeeds 71% of the time, as Claude spends a lot of time underground, which explains the higher furnace/coal progression.

Table 6.15: Research autonomous: goal distribution (aggregate)

Goal theme	Approx. share	Interpretation
`escape_trap` / forced recovery	∼11%	The single most common goal, driven by the terrain-escape guard
`escape to surface`	∼8%	Repeated attempts to get out of caves/cliffs
`escape_drowning`	∼3%	Water hazards
`replace_worn_tool`	∼2%	Tool durability guard
Crafting / mining stone tools	∼8% combined	Proactive progression
Collect food / hunt animals	∼3% combined	Food-seeking, mostly unsuccessful
All other goals	*∼65%*	Highly diverse, run-specific goals

The number of total distinct goals observed is larger than 150. The top goal accounts for only ≈ 11% of decisions, compared to ≈ 16% in the guided baseline and much higher concentration in the heuristic baseline. This confirms that removing NEXT STEP produces far more diverse, context-specific goal setting.

Table 6.16: Research autonomous: death causes

Cause	Count	% of deaths
Environmental / starvation / fall damage	6	40%
Drowning	2	13%
Hostile mobs (spider, skeleton, creeper, zombie villager)	7	47%
Total deaths	**15**	**100%**

Autonomous deaths are split roughly evenly between hostile mobs and environmental causes. Many environmental deaths happen because the agent voluntarily mines deep or explores hazardous terrain and then cannot escape. This is a different failure mode from the guided baseline, where most deaths are mob kills after leaving shelter.

Perturbation wise, the agent has similar issues as to the previous experiment. From forced death, the agent recovers within one life, while for teleportation it provides mixed results, sometimes the LLM adapts by exploring the new area, but it can also get stuck in cliffs/water and die before stabilizing. Inventory wipe showcases a major breakdown. Like the guided baseline, the agent can fall into a post-wipe death spiral, especially if it loses its crafting table and pickaxe. The difference is that Claude often re-crafts a crafting table and pickaxe more quickly, shortening the spiral.

The autonomous experiments show clear differences from the guided runs: higher progression ceiling, more diverse and context specific goals, and similar survival fragility. Claude reaches stone tools in every run and furnaces in 4/5 runs. It thinks about coal, iron, and even copper (goals the guided baseline rarely generates). Without the progression directive, the agent produces a much broader goal distribution and adapts reasoning to situation. Deaths per run are only slightly

lower than guided, and the same action-level failures (`pillar_up`, `dig_to_surface`, food acquisition) dominate. A stronger LLM improves planning, but it cannot fully compensate for unreliable low-level actions.

6.4 Cross-configuration comparison

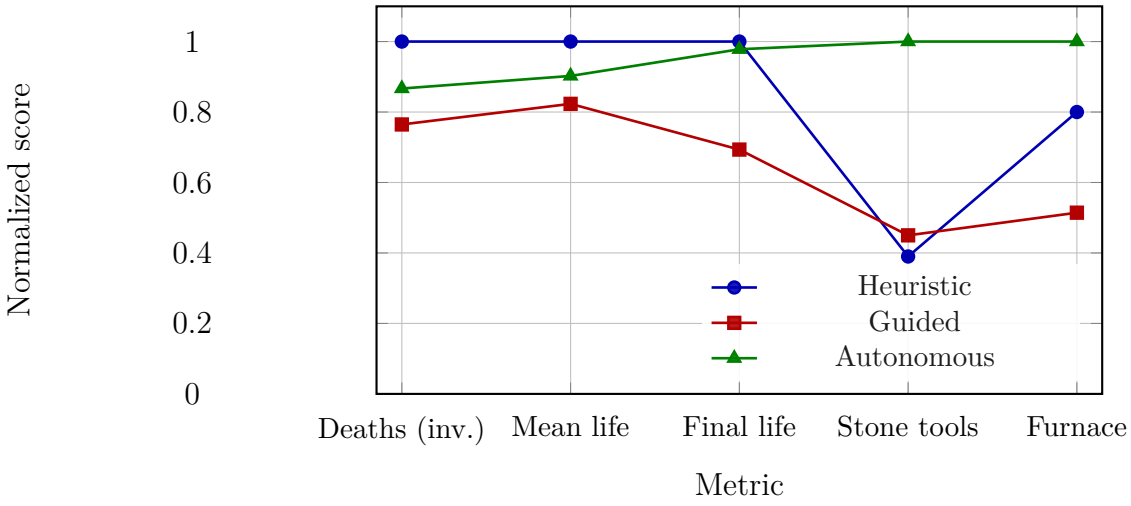

Figure 6.1: Cross-configuration comparison

The performance profile is normalized, meaning each metric is scaled so that 1 is the best value among the three configurations; deaths per run is inverted because lower is better. The autonomous profile nearly matches the heuristic baseline on survival while dominating progression; the guided baseline is intermediate on both dimensions. Autonomous per-life progression counts are cumulative across lives.

Takeaways from the comparison include survival, where the heuristic approach is the most stable, having fewer deaths and longest mean life, while the guided LLM is the most volatile. Progression metrics show that Claude outperforms on progression ceilings, 100% of runs reach stone tools and 80% reach furnaces, versus 45% and 18% for guided, and 39% and 28% for the heuristic baseline. Regarding goal diversity, autonomy is ahead as it produces the most diverse goals, guidance produces the most repetitive, guard-heavy goal distribution and the heuristic is the most stereotyped.

The experiment was designed to test whether a stronger LLM, given less explicit instruction, would behave more coherently and progress further in an open-ended survival sandbox. The results are mixed in a revealing way. The heuristic baseline is a competent but shallow survival engine. The guided LLM baseline shows that explicit instruction helps early progression but not robustness. The autonomous Claude configuration shows that more capable reasoning improves progression ceiling and behavioral diversity. A consistent pattern across all configurations is that planning exceeds execution. The LLM formulates appropriate plans, but the low level actions fail frequently enough to derail the plans. For example, none of

the configurations reaches iron or beds, and not because the autonomous model lacks the concept, Claude reasons about iron ores and beds, but the steps to obtain are too long and fragile (Find and mine coal; Craft a furnace and torches; Mine iron ore; Smelt iron ingots; Craft an iron pickaxe). Each step is dependent on the previous one and any perturbation resets the progress.

Lastly, a prominent question is whether guidance helps or hurts the reasoning process. The comparison between guided and autonomous LLM configurations suggests that explicit guidance is a double-edged sword. It improves early progression by constraining the model to a known recipe, but it also makes the agent less flexible when conditions deviate from that recipe. The autonomous agent, by contrast, generates more diverse recovery strategies, mining underground for safety, exploring to find resources, crafting a new crafting table when the old one is lost, even if those strategies do not always succeed.

6.5 Coherence and Emergent Behavior

Metrics such as survival and progression are important in order to study the ability of the agent to survive and adapt to the environment and unexpected changes. In addition, coherency and emergence of behavior are needed to provide broad understanding of the experiments.

6.5.1 Coherence

Goal diversity showcases the level of specificity of a decision taken, relative to the situation. Variety, continuity and relevance to the environment are traits that sustain coherence. Below are the results recorded:

Table 6.17: Goal diversity and intervention rates

Metric	Heuristic baseline	Guided baseline (GPT-4o-mini)	Research autonomous (Claude)
Total goals logged	1,014	1,014	635
Unique goal strings	38	68	310
Goal entropy (Shannon, bits)	3.88	4.70	7.20
Top goal share	`gather_wood` 24.4%	`build shelter` 16.1%	`escape_trap` 11.2%
Guard-override ratio	45.7%	47.2%	18.6%
Average goal streak length	2.51 loops	1.95 loops	1.17 loops
Longest single-goal streak	98 (`gather_wood`)	80 (`build shelter`)	7 (`Mine cobblestone for stone tools`)

A clear pattern is visible, that autonomy produces more diverse goals. The autonomous agent generates over four times as many unique goals as the heuristic

baseline and over four times as many as the guided baseline. At the same time, the autonomous agent receives guard overrides in fewer than one in five decisions, compared to nearly half for the other two configurations.

However, the short goal streaks in the autonomous configuration (average 1.17 loops per goal) also reveal a tendency to switch goals frequently. The model often revises its plan every loop in response to new observations. This is coherent with an agent that is actively reasoning about a changing world, but it also means many plans are abandoned before completion.

Plan completion is being recorded as the share of goals that the agent explicitly "completes" before moving on, but also how long the agent stays stuck on failing actions.

Table 6.18: Decision coherence and plan completion

Metric	Heuristic baseline	Guided baseline (GPT-4o-mini)	Research autonomous (Claude)
Average goal completion rate	2.12%	1.73%	3.85%
Average max consecutive failures	3.8	5.9	4.3
Average perturbation recovery time	966.5 s	334.0 s	276.5 s

All configurations have low completion rates, which is expected in a sandbox, as goals are open-ended and action failure is large. The autonomous agent has the highest completion rate, approximately twice the guided agent. This suggests that Claude is better at recognizing when a goal has been achieved and moving on, even if the absolute rate is still low.

Another visible pattern is the guided agent's failure streak, reflecting its tendency to get trapped in loops. The heuristic baseline has shorter failure streaks because its deterministic rules are simpler and switch earlier. The autonomous agent sits in the middle, it fails often, but its diversity of goals helps it escape loops sooner than guided.

Recovery time after perturbations is fastest for autonomous and guided, and much slower for the heuristic baseline. This confirms that LLM reasoning improves adaptation speed, even when it does not improve overall survival.

An overview of coherence over conditions is:

Table 6.19: Comparing coherence across conditions

Aspect	Heuristic	Guided	Autonomous
Goal diversity	Low	Medium	High
Plan persistence	High (stereotyped loops)	Medium (guard-driven loops)	Low (frequent switching)
Guard dependence	High	High	Low
Adaptation speed	Low	Medium	High
Emergent strategies	None	Few (mostly script-following)	Many
Thrashing risk	Low (predictable loops)	Medium	High

The heuristic baseline is the most coherent in a narrow sense: its behavior is highly predictable and it rarely thrashes. But it is coherent only because it has no real alternatives. The guided baseline is coherent when the script matches reality, but it falls apart when conditions deviate. The autonomous baseline is the least predictable and most prone to thrashing, but it is also the only one that produces genuinely novel, context-sensitive strategies.

Thrashing is a side effect of the diversity that enables coherence and emergent behavior and produces incoherence. Goal thrashing means that the autonomous agent sometimes switches goals every loop without making progress on any of them. Another issue is repeating failed actions, the agent sometimes retries them several times in a row before trying a different action. Incoherence is also generated by over-reaction to low-priority alerts, where the agent occasionally spends many loops trying to collect food when no animals are nearby, even though food is only moderately low. In addition, confusion about own state happens, in run 1/5, the agent repeatedly believed it was underground while the observation also reported exposed to open sky, leading to a long sequence of contradictory escape attempts.

Finally, the coherence analysis suggests a nuanced answer, the autonomous agent reasons sensibly about survival, progression, and exploration, but execution is fragmented, goals switch quickly, failed actions are repeated, and state confusion is common. Autonomy clearly enables emergent behaviors that guidance suppresses. The trade-off is reliability.

6.5.2 Emergent Behavior

Several behaviors that were observed were not hard-coded in actions system, guards or prompts:

Proactive night shelter The autonomous agent frequently decides to mine underground or build a shelter before night falls, based on the time observation. For example, in run 3/5 it mined into stone and crafted torches to survive the night, then continued mining for coal the next day. This is a genuine planning behavior, as the model is not merely reacting to darkness but anticipating it.

Resource and tool management The autonomous agent regularly crafts a crafting table, wooden pickaxe, stone pickaxe, and furnace in sequence, and it remembers to equip the right tool. In run 4/5 it carried stacks of cobblestone, granite, coal, and raw copper, suggesting it was stockpiling resources for future crafting. The guided baseline also tool-chains, but it does so because the progression directive tells it to.

Recovery after inventory wipe After the inventory wipe at 30 minutes, the autonomous agent often re-crafts a crafting table and pickaxe from scratch rather than entering the purely reactive loops seen in the guided baseline. In run 5/5, after losing everything, it re-established a stone pickaxe and began mining stone within a few lives. This is an emergent recovery strategy rather than a scripted one.

Exploration-driven adaptation The autonomous agent explores more purposefully. When teleported, it sometimes explores the new area, identifies resources (oak logs, stone, coal), and adjusts its plan. The guided baseline more often defaults to the scripted early-game loop regardless of terrain.

Defensive crafting In several runs, Claude crafted a stone sword proactively when it anticipated combat risk, and it sometimes placed torches or built pillars to avoid mobs. These behaviors were generated from the system prompt's survival rules but were not mapped to a fixed guard.

6.6 Comparison with Existing Approaches

The same general idea of observation-based reasoning and action selection as in the experiments is visible in ReAct[15], but applied to a smaller, less persistent environment. ReAct demonstrates the usefulness of reasoning-action loops, while this thesis shows that such loops become more difficult when actions affect future survival conditions over extended periods.

The results also provide an interesting contrast to Reflexion[9] and Self-Refine[5]. These papers propose approaches that improve performance through iterative feedback and self-critique. Despite the fact that in the current framework no reflection mechanism exists, the LLMs frequently adapted their behavior after perturbations. Although such adaptations did not always succeed, they suggest that contextual reasoning alone can support a limited degree of behavioral adaptation.

The experiments share several similarities with Generative Agents[6]. Both studies demonstrate that language models can maintain behavioral continuity over long periods. However, the source of behavior is highly different, while Generative Agents focus on social interactions, memories, and routines, the present work places the agent within a survival environment dominated by resource acquisition, environmental hazards, and progression. The results therefore broaden the study of long-term LLM behavior beyond social simulation.

The most similar work, Voyager[13] exposes exploration, resource gathering and progression in the same simulated environment, but differs by focusing on skill

acquisition and code generation, contrasting to the current thesis which deliberately fixes the available action space and removes explicit objectives. The results show that meaningful progression can still occur without externally defined goals, although the achieved progression remains substantially lower than that reported by Voyager.

Tool-oriented approaches such as Toolformer [8] and SayCan [1] demonstrate that language models can successfully interact with external systems and select appropriate actions. The present work extends these ideas to persistent environments, where decisions influence future states over extended periods. The experiments indicate that selecting actions is only one component of autonomous behavior. Reliable execution of those actions is equally important.

The comparison with reinforcement learning approaches[11] and optimization-driven systems such as AlphaGo[10] provides an important contrast. Reinforcement learning systems optimize behavior through explicit rewards, while the proposed framework intentionally removes reward signals and predefined objectives. Although the resulting agents are less efficient and less reliable than reward-optimized systems, they exhibit greater behavioral diversity and more varied goal formation. The experiments therefore support the idea that language-based autonomy represents a complementary paradigm rather than a replacement for reinforcement learning.

Finally, the observed goal diversity and decision making provide evidence for emergent behavior as described by Goldstein[4]. The autonomous configuration generated more than 150 distinct goals, many of which were not explicitly encoded by the framework. The agent adapted its objectives according to environmental conditions, available resources, and previous failures. Although limitations prevent strong conclusions regarding emergence, the experiments suggest that complex behavioral patterns may arise even in the absence of reward optimization or externally imposed objectives.

Chapter 7

Conclusion

The objective of the experiments conducted is to investigate the autonomous behavior of an LLM in a simulated environment where the focus falls on how the model perceives context, decision making and adapting rather than optimization of task performance.

Contrary to initial guesses, a stronger LLM does not perform the best for survival metrics. The heuristic baseline proved to be the most reliable configuration. Regarding the LLM configurations death frequency, the guided gpt-4o-mini configuration averaged 3.4 deaths per run with a mean life of 13.5 minutes, while the autonomous Claude Sonnet configuration averaged 3.0 deaths per run with a mean life of 14.8 minutes. This result demonstrates that scripted survival priorities provide a robust floor, and that high-level reasoning does not automatically translate into longer survival when low-level execution remains fragile. Despite the reliability of the heuristic baseline for survival, for progression the LLM configurations distinguished themselves. The heuristic baseline reached stone tools in 39 % of lives and a furnace in 28 %, never advancing to iron tools or beds. The guided GPT-4o-mini baseline improved slightly on stone-tool reach (45 % of lives) but actually regressed on furnace acquisition (18%), and likewise failed to reach iron or beds. The autonomous Claude configuration, however, reached stone tools in every run and crafted a furnace in 4 of 5 runs. This is a clear improvement in progression ceiling, indicating that a more capable model, when freed from explicit instruction, can formulate and execute longer tool-chains and resource-gathering plans.

The most notable difference between the three configurations was in the diversity of declared goals. The heuristic baseline produced 38 unique goal strings across 1,014. The guided baseline produced 68 unique goals. The autonomous Claude configuration produced 310 unique goals. This increase in unique goal strings, combined with a guard-override ratio of only 18.6 % compared to roughly 47 % for the other two configurations, strongly suggests that autonomy enables context-sensitive decision-making rather than reactive script-following.

Both LLM configurations recovered from perturbations substantially faster than the heuristic baseline. Average perturbation recovery time was 966.5 seconds for the heuristic engine, 334.0 seconds for the guided baseline, and 276.5 seconds

for the autonomous configuration. The autonomous agent in particular showed emergent re-bootstrapping behavior after inventory wipe, re-crafting crafting tables and pickaxes from scratch and resuming resource gathering. This confirms that language-model reasoning confers a measurable advantage in adaptation, even when it does not improve raw survival time.

Emergent behavior was visible for the autonomous agent, which anticipated nightfall and prepared for it, cached resources, equipped tools and adapted after perturbations. These behaviors were not scripted, they emerged from the interaction between the model's pre-trained reasoning, the structured observation prompt, and the survival-oriented system instructions. In this sense, the experiment answers its central question: LLMs can generate coherent, multi-step goals without explicit task instruction.

This thesis set out to investigate whether Large Language Models can behave coherently and autonomously in an open-ended survival environment when given only survival scaffolding and no explicit progression objectives. The empirical evidence supports a qualified affirmative. A sufficiently capable LLM, operating without scripted milestones, generates more diverse, context-sensitive, and strategically interesting goals than either a deterministic heuristic baseline or a heavily scaffolded weaker model. It reaches higher progression tiers and recovers faster from perturbations. The broader lesson is that embodied LLM agents cannot be evaluated solely as reasoning systems; they must be evaluated as **integrated perception-reasoning-execution systems**. A model that can articulate a sensible plan to smelt iron is of little use if it cannot reliably escape the cave in which it mines the ore. Future progress in this domain will therefore depend not only on more capable language models, but also on tighter coupling between what the model intends and what the agent can mechanically execute. The framework and dataset established in this thesis provide a reproducible foundation for that continued investigation.

Bibliography

[1] Michael Ahn, Anthony Brohan, Noah Brown, Yevgen Chebotar, Omar Cortes, Byron David, Chelsea Finn, Chuyuan Fu, Keerthana Gopalakrishnan, Karol Hausman, Alex Herzog, Daniel Ho, Jasmine Hsu, Julian Ibarz, Brian Ichter, Alex Irpan, Eric Jang, Rosario Jauregui Ruano, Kyle Jeffrey, Sally Jesmonth, Nikhil J. Joshi, Ryan Julian, Dmitry Kalashnikov, Yuheng Kuang, Kuang-Huei Lee, Sergey Levine, Yao Lu, Linda Luu, Carolina Parada, Peter Pastor, Jornell Quiambao, Kanishka Rao, Jarek Rettinghouse, Diego Reyes, Pierre Sermanet, Nicolas Sievers, Clayton Tan, Alexander Toshev, Vincent Vanhoucke, Fei Xia, Ted Xiao, Peng Xu, Sichun Xu, Mengyuan Yan, and Andy Zeng. Do as i can, not as i say: Grounding language in robotic affordances. In *Proceedings of the 7th Conference on Robot Learning (CoRL 2023)*, 2023. URL `https://arxiv.org/abs/2204.01691`.

[2] Bowen Baker, Ingmar Kanitscheider, Todor Markov, Yi Wu, Glenn Powell, Bob McGrew, and Igor Mordatch. Emergent tool use from multi-agent autocurricula. *arXiv preprint arXiv:1909.07528*, 2019. URL `https://arxiv.org/abs/1909.07528`.

[3] Tom B. Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, Sandhini Agarwal, Ariel Herbert-Voss, Gretchen Krueger, Tom Henighan, Rewon Child, Aditya Ramesh, Daniel M. Ziegler, Jeffrey Wu, Clemens Winter, Christopher Hesse, Mark Chen, Eric Sigler, Mateusz Litwin, Scott Gray, Benjamin Chess, Jack Clark, Christopher Berner, Sam McCandlish, Alec Radford, Ilya Sutskever, and Dario Amodei. Language models are few-shot learners. In *Advances in Neural Information Processing Systems 33 (NeurIPS 2020)*, 2020. URL `https://arxiv.org/abs/2005.14165`.

[4] Jeffrey Goldstein. Emergence as a construct: History and issues. *Emergence*, 1(1):49–72, 1999. doi: 10.1207/s15327000em0101_4.

[5] Aman Madaan, Niket Tandon, Prakhar Gupta, Skyler Hallinan, Luyu Gao, Sarah Wiegreffe, Uri Alon, Nouha Dziri, Shrimai Prabhumoye, Yiming Yang, Shashank Gupta, Bodhisattwa Prasad Majumder, Katherine Hermann, Sean Welleck, Amir Yazdanbakhsh, and Peter Clark. Self-refine: Iterative refinement with self-feedback. *Advances in Neural Information Processing Systems*, 36, 2023. URL `https://arxiv.org/abs/2303.17651`.

[6] Joon Sung Park, Joseph C. O'Brien, Carrie J. Cai, Meredith Ringel Morris, Percy Liang, and Michael S. Bernstein. Generative agents: Interactive simulacra of human behavior. In *Proceedings of the 36th Annual ACM Symposium*

on User Interface Software and Technology (UIST '23), pages 1–22, 2023. doi: 10.1145/3586183.3606763. URL https://dl.acm.org/doi/10.1145/3586183.3606763.

[7] Stuart J. Russell and Peter Norvig. *Artificial Intelligence: A Modern Approach*. Pearson, Hoboken, 4 edition, 2021. ISBN 9780134610993.

[8] Timo Schick, Jane Dwivedi-Yu, Roberto Dessì, Roberta Raileanu, Maria Lomeli, Luke Zettlemoyer, Nicola Cancedda, and Thomas Scialom. Toolformer: Language models can teach themselves to use tools. *Advances in Neural Information Processing Systems*, 36, 2023. URL https://arxiv.org/abs/2302.04761.

[9] Noah Shinn, Federico Cassano, Edward Berman, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. Reflexion: Language agents with verbal reinforcement learning. *Advances in Neural Information Processing Systems*, 36, 2023. URL https://arxiv.org/abs/2303.11366.

[10] David Silver, Aja Huang, Chris J. Maddison, Arthur Guez, Laurent Sifre, George van den Driessche, Julian Schrittwieser, Ioannis Antonoglou, Veda Panneershelvam, Marc Lanctot, Sander Dieleman, Dominik Grewe, John Nham, Nal Kalchbrenner, Ilya Sutskever, Timothy Lillicrap, Madeleine Leach, Koray Kavukcuoglu, Thore Graepel, and Demis Hassabis. Mastering the game of go with deep neural networks and tree search. *Nature*, 529(7587):484–489, 2016. doi: 10.1038/nature16961.

[11] Richard S. Sutton and Andrew G. Barto. *Reinforcement Learning: An Introduction*. MIT Press, Cambridge, MA, 2 edition, 2018. ISBN 9780262039246. URL https://incompleteideas.net/book/the-book-2nd.html.

[12] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems 30 (NeurIPS 2017)*, 2017. URL https://arxiv.org/abs/1706.03762.

[13] Guanzhi Wang, Yuqi Xie, Yunfan Jiang, Ajay Mandlekar, Chaowei Xiao, Yuke Zhu, Linxi Fan, and Anima Anandkumar. Voyager: An open-ended embodied agent with large language models. *arXiv preprint arXiv:2305.16291*, 2023. URL https://arxiv.org/abs/2305.16291.

[14] Jason Wei, Yi Tay, Rishi Bommasani, Colin Raffel, Barret Zoph, Sebastian Borgeaud, Dani Yogatama, Maarten Bosma, Denny Zhou, Donald Metzler, Ed H. Chi, Tatsunori Hashimoto, Oriol Vinyals, Percy Liang, Jeff Dean, and William Fedus. Emergent abilities of large language models. *Transactions on Machine Learning Research*, 2022. URL https://openreview.net/forum?id=yzkSU5zdwD.

[15] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. React: Synergizing reasoning and acting in language models. *arXiv preprint arXiv:2210.03629*, 2022. URL https://arxiv.org/abs/2210.03629.

Statement of Scientific integrity

I declare that this BSc thesis is an original report and represents my own work, conducted under supervision of Lect. Dr. Cristian Cira. All research methods, data collection, analysis and interpretation presented in this thesis are accurately reported and all sources have been cited appropriately where I have made use of ideas and/or visuals of others.

I used generative AI during the writing and research process. The details are in the Appendix 7.

Appendix - Generative AI Tools Usage

An overview how generative AI Tools are used in this thesis is presented in Table 7.1.

AI Tool	Use case	Used in	Remarks
ChatGPT 5.5	Verify citations properly attributed	Entire thesis	Prompts in section 7.1

Table 7.1: Overview of generative AI Usage

7.1 ChatGPT Details

For current usecase:

Purpose of use Robustness check of citations to ensure no misattributions or lack of attributions: As in - I prompted chat GPT to verify which paragraphs still needed citation number tag (i.e [15]) and if there are any that were using wrong tags. A

Method (which promtps and for which parts of the thesis) "you are a latex parser and analyzer.

do a robustness check of all citations, launch subagents as needed for deeper searches in cited papers

your final output includes a report of cited sources against content in the paper to see if there are any mismatches that need correction, or any paragraphs not tagged with citation.

DO NOT GENERATE IMAGES UNLESS EXPLICITLY ASKED TO"

Used for citation validation after Thesis was finished

How the output has been examined/verified Cross check

How the output is implemented/used Citations fixed as needed and content review if misinterpreted.

Prompt output:

Full Scan Result
I checked all citation occurrences in main_file_template_bachelor_masters_english.tex. No missing keys, no unused .bib entries, and no citation commands in cross_config_plot.tex / decisionLoop.tex.

The actual problems are:

Type Count Wrong citation key 1 Claims needing extra citation 6 Slight source misinterpretation 2 Everything else OK

1. Wrong Citation Key At the Voyager paragraph, this is definitely wrong. Voyager is wang2023voyager, not park2023generative. Voyager is the Minecraft agent with automatic curriculum, skill library, and iterative prompting.

Replace:

Voyager [6] presents an LLM-based agent operating in an sandbox, open-world game, Minecraft.

with:

Voyager [13] presents an LLM-based agent operating in an sandbox, open-world game, Minecraft.

2. Add Missing Citations Only

Reasoning + tool use needs both ReAct and Toolformer. ReAct supports reasoning/action loops; Toolformer supports tool/API use.

Whilst modern day LLMs can perform various tasks, ranging from reasoning to using tools [15, 8], their long-term autonomous behavior and consistency remain largely a mystery.

Interpretability claim should include ReAct, not only SayCan:

Each agent excels in their own environments, RL are proficient in habitats with clear definition of objectives and rewards, while language-based agents perform well in open-ended settings with ambiguous or emergent goals. Furthermore, language-based agents offer greater interpretability, as their decision processes can be expressed and analyzed through natural language explanations [15, 1].

Memory/reflection/self-improvement needs Reflexion + Self-Refine, not only Self-Refine:

Furthermore, while recent work introduced memory mechanisms, reflection and self-improvement [9, 5], they are evaluated in iterative task settings where only performance improvements can be measured.Not enough attention has been given to how such mechanisms influence behavior in goal-free, continuous environments, where there is no explicit notion of success or failure. Similarly, reinforced learning approaches provide a baseline for emergent behavior, but their reliance on reward shaped agent dynamics [11, 2], which makes them unsuitable for studying unconstrained decision-making processes

Simulated environments / games as AI testbeds needs more than Sutton:

Simulated environments and video games have long been used as testbeds for artificial intelligence research, as they provide controlled, but expressive environments [2, 10, 13]. Classical approach rely on reinforced learning, the agent optimizes explicit reward functions through repeated interaction[11]. Recently language models are used as an alternative controller, being capable of interpreting environment descriptions and selecting actions [13].

Embodied / simulated LLM sentence needs Park/Voyager before Baker:

Recent work combines LLMs with simulated embodiment [6, 13]. Multi-agent extensions of this approach enable agents to interact - compete or cooperate - in a shared environment [6, 2]. Studies on emergent tool use in multi-agent reinforcement learning systems demonstrate that complex coordination strategies can arise without explicit programming [2].

Self-improving agents with memory/self-feedback needs Reflexion too:

Recent language-based systems explore self-improving agents that iteratively refine their behavior using memory and self-generated feedback [9, 5]. However, many such systems rely on extensive scaffolding, external evaluators, or explicit improvement objectives, making it difficult to isolate and analyze the intrinsic autonomy of the language model itself.

AlphaGo / reward systems should cite Sutton too:

Each paradigm involves trade-offs in terms of adaptability, explainability, and generalization. Unlike reinforcement learning agents or search-based systems such as AlphaGo, which rely on explicit optimization objectives and reward signals [11, 10], LLM-driven agents operate through implicit reasoning derived from contextual input.

3. Slight Misinterpretations

SayCan does not really use “symbolic planning constraints”; it uses language-model scoring plus affordance/feasibility grounding through pretrained skills. Minimal fix:

Planning-capable systems further extend this paradigm. SayCan integrates language-model likelihoods with affordance-based feasibility constraints to select executable actions in robotic domains [1]. These approaches position the language model as a high-level planner rather than a fully autonomous agent embedded in a persistent environment.

This one is slightly too strong:

Even in open-ended simulations, Generative Agents [6], agent behavior is guided, be it by implicit social objectives or scripted routines.

Park supports memory, reflection, planning, social interaction, and a sandbox town, but "scripted routines" can sound like the behaviors are hardcoded, which is not quite fair. Minimal fix:

Even in open-ended simulations, Generative Agents [6], agent behavior is guided, be it by implicit social objectives or structured routines.

Everything Else The remaining citations are source-compatible: wei2022emergent, vaswani2017attention, brown2020fewshot, russell2021aima, sutton2018rl, goldstein1999emergence, yao2022react, shinn2023reflexion, park2023generative, wang2023voyager, schick2023toolformer, ahn2023saycan, baker2019emergent, madaan2023selfrefine, and silver2016alphago are all used for claims they can reasonably support after the small fixes above.